NGUYỄN THẾ HUY

KIẾN TRÚC ỨNG DỤNG WEB

CÁC NGUYÊN TẮC CỐT LÕI ĐỂ ỨNG DỤNG WEB DỄ BẢO TRÌ VÀ MỞ RỘNG

https://huynt.dev

Lời mở đầu

Chào bạn, và cám ơn bạn đã đọc ebook này của mình.

Mình là Huy, hiện tại mình là một Technical Leader với 10 năm kinh nghiệm.

Ebook này của mình nhằm cung cấp cho bạn những khái niệm cốt lõi về một kiến trúc ứng dụng web: từ việc tách rời code nghiệp vụ và code hạ tầng, cho đến việc triển khai Hexagon Architecture trong hệ thống.

Dù bạn là người mới bắt đầu hay đã có kinh nghiệm, mình tin những kiến thức trong ebook này sẽ giúp bạn rất nhiều trong việc nâng cấp codebase của mình, làm quen với những kiến trúc ứng dụng hiệu quả, đảm bảo rằng bạn luôn có phương án thiết kế ngay cả với dự án mới hay các dự án đã “legacy”.

Chúc bạn có những giây phút thú vị, bổ ích khi đọc ebook.

Mục tiêu và tầm quan trọng của việc tách biệt core code & infrastructure code

Mục tiêu của việc tách biệt code lõi và code hạ tầng

Trong phát triển ứng dụng web, code lõi (core code) được hiểu là phần code chứa logic nghiệp vụ thuần túy của ứng dụng, bao gồm những quy tắc, thao tác và xử lý phản ánh đúng mô hình hoạt động của lĩnh vực (domain) mà ứng dụng phục vụ. Ngược lại, code hạ tầng (infrastructure code) là phần code đóng vai trò kết nối, tương tác giữa logic lõi của ứng dụng với các hệ thống bên ngoài như cơ sở dữ liệu, third party, hệ thống file, framework web, v.v . Mục tiêu của việc tách biệt hai loại code này là để chúng không trộn lẫn trong cùng một thành phần, giúp thiết kế hệ thống rõ ràng hơn và dễ dàng quản lý hơn. Cả hai loại code đều quan trọng như nhau trong ứng dụng, nhưng nếu chúng ở chung trong một layer hoặc module, sẽ phát sinh nhiều vấn đề về bảo trì và phát triển.

Mục tiêu chính của kiến trúc tách biệt core code và infrastructure code là tạo ra một ranh giới rõ ràng giữa phần logic nghiệp vụ cốt lõi và phần triển khai kỹ thuật bên ngoài. Khi đạt được điều này, chúng ta có thể phát triển phần domain (nghiệp vụ) một cách độc lập, tập trung, mà không bị ảnh hưởng bởi chi tiết cơ sở dữ liệu, framework hay các dịch vụ bên ngoài. Đồng thời, việc tách biệt giúp xây dựng một nền tảng kỹ thuật vững chắc để phát triển theo hướng domain-driven (thiết kế hướng nghiệp vụ) và dễ dàng áp dụng các thực hành như test-first hay TDD (Test-Driven Development) .

Nói cách khác, code lõi được viết sao cho thể hiện trực tiếp yêu cầu nghiệp vụ (ví dụ: “tạo đơn hàng mới”, “tính tổng số tiền”) một cách rõ ràng, còn code hạ tầng sẽ đảm nhiệm chi tiết cách thức để lưu trữ dữ liệu, giao tiếp mạng, đọc/ghi file, v.v. Mục tiêu là khi đọc phần core code, developer thấy được câu chuyện nghiệp vụ đang diễn ra, thay vì bị che giấu trong các đoạn code nặng tính mô tả kỹ thuật. Ví dụ, thay vì trong hàm xử lý đơn hàng có những câu lệnh SQL insert dữ liệu, chúng ta muốn hàm đó chỉ gọi một phương thức “lưu đơn hàng” ở mức tổng quát. Chi tiết thực hiện “lưu” (dùng SQL hay gọi API, v.v.) sẽ do lớp hạ tầng đảm nhận. Bằng cách này, luồng xử lý nghiệp vụ trở nên dễ hiểu hơn và ít phụ thuộc vào quyết định công nghệ cụ thể.

Tóm lại, mục tiêu của việc tách biệt là để mỗi phần code có một trách nhiệm rõ ràng: phần core tập trung giải quyết vấn đề nghiệp vụ, phần infrastructure lo việc tương tác môi trường ngoài.

Điều này hướng tới mã nguồn trong sáng, dễ đọc, và một kiến trúc linh hoạt sẵn sàng cho những thay đổi trong tương lai.

Những vấn đề khi code lõi và code hạ tầng trộn lẫn

Khi code lõi và code hạ tầng không được tách riêng, dự án sẽ gặp phải nhiều vấn đề phức tạp theo thời gian. Dưới đây là một số vấn đề phổ biến:

- **Khó nắm bắt logic nghiệp vụ:** Khi xem một đoạn code trộn lẫn truy cập cơ sở dữ liệu, gọi API, xử lý request HTTP cùng với tính toán nghiệp vụ, rất khó để hiểu câu chuyện nghiệp vụ chính đang diễn ra là gì. Các chi tiết triển khai kỹ thuật (như truy vấn SQL, thao tác file, v.v.) che mờ ý nghĩa của các bước nghiệp vụ. Ví dụ, trong một hàm xử lý "đặt mua e-book", thay vì thấy rõ các bước như "Lấy giá ebook", "Tính tổng tiền", "Lưu đơn hàng", ta lại thấy một mớ lệnh SQL và xử lý mảng, đòi hỏi người đọc phải "dịch ngược" các đoạn code kỹ thuật để suy ra được ý nghĩa nghiệp vụ của chúng . Code bị "trộn tầng" như vậy làm giảm tính minh bạch của logic ứng dụng.
- **Phụ thuộc chặt vào công nghệ, khó thay đổi về sau:** Khi logic nghiệp vụ gắn chặt với chi tiết hạ tầng, mọi thay đổi công nghệ sẽ ảnh hưởng lan rộng. Chẳng hạn, nếu code xử lý đơn hàng gọi trực tiếp SQL MySQL, việc chuyển sang dùng một cơ sở dữ liệu NoSQL hoặc thậm chí chỉ thay đổi loại cơ sở dữ liệu cũng đòi hỏi chỉnh sửa rất nhiều chỗ trong logic . Tương tự, nếu luồng xử lý gắn với việc nhận dữ liệu từ form web, sau này muốn tái sử dụng logic đó cho một API JSON hoặc một giao diện dòng lệnh sẽ gặp nhiều khó khăn . Sự kết hợp chặt chẽ giữa phần lõi và công nghệ bên ngoài làm giảm tính linh hoạt và khả năng mở rộng của hệ thống.
- **Khó kiểm thử tự động:** Code chứa nhiều phụ thuộc bên ngoài (như kết nối DB, gọi ra API ngoài, đọc file) rất khó để viết unit test hoặc automation test một cách độc lập. Để chạy được những đoạn code này, môi trường test phải có sẵn DB với bảng dữ liệu, third party phải hoạt động, file phải tồn tại... dẫn đến việc test trở nên phức tạp, chậm chạp hoặc không ổn định. Ngược lại, nếu tách phần tương tác ngoài ra, code lõi có thể chạy trong cô lập hoàn toàn, không cần DB, không cần tương tác network - giúp viết test dễ dàng và nhanh chóng hơn rất nhiều . Việc trộn lẫn làm chúng ta mất đi khả năng kiểm thử hiệu quả, dẫn đến ngại viết test và hậu quả là ứng dụng thiếu các bộ test bảo vệ.
- **Không bảo đảm tính nhất quán dữ liệu:** Khi logic nghiệp vụ và thao tác dữ liệu nằm rải rác, việc đảm bảo dữ liệu hợp lệ trở nên khó khăn. Ví dụ, nếu mọi nơi trong code trực tiếp

chèn dữ liệu vào database, rất dễ xảy ra trường hợp dữ liệu sai (như email sai định dạng, số lượng âm, tính toán sai tiền) được đưa thẳng vào database mà không qua khâu kiểm tra hợp lệ. Trong một code base trộn lẫn, không có một lớp đối tượng trung tâm nào chịu trách nhiệm bảo vệ tính nhất quán của dữ liệu domain cả. Điều này dẫn đến nguy cơ lỗi nghiệp vụ cao (ví dụ lưu đơn hàng với email không hợp lệ, số tiền không đúng) mà không được phát hiện kịp.

- **Bảo trì khó khăn và dễ lỗi:** Code trộn lẫn thường vi phạm nguyên tắc ***Separation of Concerns (tách bạch mối quan tâm)***. Mỗi thay đổi nhỏ có thể buộc developer phải chỉnh sửa ở nhiều nơi - ví dụ đổi lược đồ cơ sở dữ liệu, phải tìm và sửa mọi câu SQL rải rác trong code nghiệp vụ. Việc này không những mất công mà còn dễ bỏ sót chỗ cần thay, phát sinh lỗi ngầm. Mặt khác, code trộn lẫn thường lặp lại nhiều đoạn tương tự (vì thiếu ***abstraction***), dẫn đến trùng lặp code và tăng khả năng sai sót. Nhìn chung, khi core code và infrastructure code không tách biệt, mã nguồn sẽ "bốc mùi" (code smell): khó đọc, khó hiểu, khó đổi mới, khó kiểm thử. Dự án sẽ dễ rơi vào tình trạng "legacy" sớm, nghĩa là chỉ sau một thời gian ngắn phát triển đã trở nên cồng kềnh, khó cải tiến, mỗi lần sửa lỗi hay thêm tính năng đều nơm nớp lo sợ ảnh hưởng dây chuyền.

Lợi ích của việc tách biệt core code và infrastructure code

Trái ngược với viễn cảnh u ám kể trên, nếu chúng ta thực hiện đúng việc phân tách core code và infrastructure code, ứng dụng sẽ có nhiều lợi ích lâu dài:

- **Hỗ trợ phát triển hướng nghiệp vụ - hướng miền (DDD):** Khi phần code lõi được tách riêng, chúng ta có thể tập trung mô hình hóa domain (miền nghiệp vụ) một cách rõ ràng, nhất quán. Điều này rất phù hợp với triết lý Domain-Driven Design: thiết kế phần mềm xoay quanh domain. Toàn bộ logic quan trọng được gói gọn trong domain model (các ***entity***, ***value object***…), cho phép nhóm phát triển và chuyên gia nghiệp vụ (domain experts) cùng hiểu rõ và tiến hóa mô hình domain mà không bị vướng bởi chi tiết kỹ thuật. Việc tách biệt hạ tầng khỏi logic lõi giúp mô hình domain có thể phát triển một cách "domain-driven" thực thụ .
- **Dễ dàng phát triển theo hướng kiểm thử (TDD):** Code lõi tách biệt hạ tầng sẽ dễ kiểm thử hơn rất nhiều, dẫn đến khả năng áp dụng Test-Driven Development hiệu quả. Bạn có thể viết các unit test nhanh cho phần logic domain (do không phụ thuộc DB, network …),

đồng thời các test này ổn định và chạy rất nhanh so với test tích hợp liên quan đến framework hoặc cơ sở dữ liệu nặng nề . Khi việc viết test trở nên thuận tiện, đội ngũ sẽ tự tin hơn khi refactor code và thêm tính năng, vì đã có test cover.

- **Minh bạch và dễ hiểu hơn:** Như đã đề cập, tách biệt sẽ làm code trong phần core trở nên "sạch" và tập trung vào cái gì làm thay vì làm như thế nào. Các bước nghiệp vụ cấp cao được thể hiện rõ bằng tên phương thức hướng vào nghiệp vụ (vd: `orderRepository->save($order)` thể hiện ý nghĩa "lưu đơn hàng" thay vì một loạt lệnh insert SQL). Nhờ đó, người đọc code hay bảo trì có thể nắm bắt nhanh dòng chảy nghiệp vụ, tài liệu hóa được yêu cầu phần mềm ngay trong mã nguồn. Code rõ ràng cũng đồng nghĩa với ít bug hơn do lập trình viên ít hiểu lầm yêu cầu hơn.
- **Tăng khả năng tái sử dụng và linh hoạt mở rộng:** Khi phần core không "dính" cụ thể vào một framework hay cơ sở dữ liệu nào, chúng ta có thể thay thế hoặc nâng cấp hạ tầng mà không ảnh hưởng đến logic chính. Ví dụ, nếu cần chuyển từ MySQL sang PostgreSQL, hay thậm chí sang một dịch vụ lưu trữ cloud khác, phần lớn core code không cần thay đổi - chỉ cần cung cấp implementation hạ tầng mới tương ứng. Tương tự, nếu muốn thêm interface API bên cạnh giao diện web, ta có thể tái sử dụng các dịch vụ và logic domain y nguyên, chỉ viết thêm lớp adapter cho giao diện mới. Sự tách biệt đảm bảo domain code không bị xáo trộn khi công nghệ bên ngoài thay đổi. Trên thực tế, nếu ứng dụng có vòng đời dài hơn 2 năm thì việc decouple khỏi hạ tầng là điều rất đáng để áp dụng vì sớm muộn các công nghệ xoay quanh (framework, database, dịch vụ ngoài) sẽ thay đổi với tốc độ khác so với domain . Nhờ decoupling, code lõi quý giá của bạn sẽ ít bị ảnh hưởng bởi những biến động công nghệ đó - mỗi bên có thể phát triển theo tốc độ riêng .
- **Dễ bảo trì, nâng cấp framework:** Vì core code không phụ thuộc trực tiếp vào framework, nên khi cần nâng cấp phiên bản framework hoặc thay thế thư viện, phạm vi ảnh hưởng sẽ được khoanh vùng ở tầng infrastructure. Chẳng hạn, nâng cấp Laravel lên bản major mới sẽ ít "đau đớn" hơn nếu phần lớn logic nghiệp vụ của bạn không viết dính chặt vào các chức năng đặc thù của phiên bản cũ. Tương tự, thay driver kết nối DB, đổi dịch vụ thanh toán… sẽ "ít tốn kém" hơn nhiều so với trường hợp code core và hạ tầng đang trộn lẫn . Tóm lại, tính ổn định và khả năng bảo trì của ứng dụng được tăng cường. Nếu một thay đổi bên ngoài xảy ra, bạn sẽ biết ngay cần sửa ở đâu - thường là trong các lớp adapter hạ tầng - thay vì dò tìm khắp code base.
- **Chuẩn hóa kiến trúc và thuận lợi cho teamwork:** Khi áp dụng triệt để phân tách, cấu trúc dự án thường hình thành các layer chuẩn: Layer Domain (chứa core code), Layer

Application (chứa dịch vụ ứng dụng, use case), Layer Infrastructure (chứa code truy xuất dữ liệu, tích hợp hệ thống ngoài). Kiến trúc nhiều lớp kết hợp với mô hình Ports và Adapters (Hexagonal Architecture) có thể được thiết lập một cách đơn giản và dễ dàng sau khi ta decouple code thành core và infrastructure . Điều này tạo ra ngôn ngữ chung và các quy tắc rõ ràng cho team khi thêm tính năng: ai cũng biết nên đặt code ở layer nào, phụ thuộc một chiều ra sao. Kết quả là việc onboard người mới dễ dàng hơn, code nhất quán và team phối hợp hiệu quả hơn.

Tất cả những lợi ích trên góp phần tạo nên một mã nguồn sạch và kiến trúc bền vững. Thậm chí, dù dự án của bạn ban đầu có nhỏ, việc phân tách ngay từ đầu sẽ kéo dài tuổi thọ của ứng dụng lâu nhất có thể. Mọi phần mềm rồi cũng có độ phức tạp tăng dần và có nguy cơ trở nên hỗn độn, nhưng nếu bạn áp dụng những nguyên tắc tách biệt core/infrastructure, "độ rối" sẽ tăng chậm hơn đáng kể, cho phép bạn kéo dài tuổi thọ dự án và thích ứng với yêu cầu mới nhanh hơn .

Giải pháp tổng quan để tách biệt core code và infrastructure code

Làm thế nào để đạt được sự tách biệt? Về mặt kiến trúc, mục tiêu này thường được hiện thực hóa thông qua các lớp (layer) và các mẫu thiết kế phù hợp. Một số nguyên tắc và kỹ thuật tổng quan bao gồm:

- **Sử dụng Dependency Injection và Inversion of Control:** Thay vì để code lõi tự mình khởi tạo hoặc gọi trực tiếp các thành phần hạ tầng, ta đảo ngược sự phụ thuộc: phần core sẽ định nghĩa các giao diện (interface) mà nó cần (ví dụ interface `OrderRepository` để lưu đơn hàng, `interface Mailer` để gửi email...). Phần infrastructure sẽ cung cấp implementation cụ thể cho các interface đó (ví dụ `MySQLOrderRepository` , `SendGridMailer` ...). Tại runtime, ta inject implementation hạ tầng vào chỗ code core thông qua constructor hoặc thông qua container của framework. Nhờ đó, core code chỉ biết đến abstraction, không biết chi tiết cụ thể bên ngoài là gì. Đây là ứng dụng nguyên lý Dependency Inversion (DIP) và là chìa khóa để tách core/infra.
- **Phân tầng kiến trúc rõ ràng:** Code được tổ chức thành các tầng như đã đề cập (Domain, Application, Infrastructure). **Tầng Domain** chứa các ***Entity, Value Object, Domain Service...*** thuần túy, không dính dáng framework. **Tầng Application** điều phối các use case, phối hợp domain với hạ tầng (qua interface): thường chính là các Application

Service hay service lớp ứng dụng. **Tầng Infrastructure** chứa các Adapter cụ thể: ví dụ lớp repository cụ thể dùng Eloquent, lớp gửi email SMTP, lớp controller framework, v.v. Luồng phụ thuộc một chiều: Domain không phụ thuộc ai, Application phụ thuộc Domain, Infrastructure phụ thuộc cả Domain (để biết interface) và có thể phụ thuộc framework. Kiến trúc này thường gọi là layered architecture. Nó đảm bảo core code nằm ở trung tâm và hạ tầng ở rìa, đúng tinh thần "onion architecture" hay "hexagonal architecture".

- **Áp dụng các mẫu thiết kế thích hợp:** Nhiều design pattern kinh điển hỗ trợ việc tách biệt này. Ví dụ: **Repository patter**n (kho lưu trữ) giúp trừu tượng hóa việc truy xuất và lưu trữ dữ liệu domain, ngăn cách giữa domain và cơ sở dữ liệu. **Domain Model** pattern giúp tập trung logic nghiệp vụ vào các đối tượng domain thay vì rải rác hoặc dùng cấu trúc bảng thuần. **Dependency Injection** (thực chất là một kỹ thuật hơn là pattern) giúp cấu hình các phụ thuộc. Factory pattern đôi khi dùng để tạo ra đối tượng thay vì gọi trực tiếp hàm của thư viện bên ngoài (ví dụ tạo đối tượng thời gian hiện tại thông qua một `Clock interface`, thay vì gọi `new DateTime()` trong core code - giúp test dễ dàng và tách biệt với hệ thống thời gian).
- **Che giấu chi tiết hạ tầng sau các abstract interface:** Mọi tương tác ra bên ngoài (IO, DB, HTTP request, file, v.v.) nên được đóng gói sau một lớp trừu tượng. Code core chỉ gọi phương thức của lớp trừu tượng đó, không quan tâm cách nó thực hiện. Ví dụ, thay vì trong code domain gọi trực tiếp PDO để query DB, ta định nghĩa một interface `OrderRepository` với method `save(Order $order)`. Triển khai cụ thể `MySqlOrderRepository` sẽ dùng PDO hay Query Builder gì đó để lưu, nhưng code domain hoàn toàn không biết – nó chỉ biết là "đơn hàng đã được lưu". Bằng cách che giấu chi tiết, ta đạt được sự đa hình: có thể có nhiều triển khai repository khác nhau (DB khác, hay giả lập in-memory để test) mà domain code không đổi gì.
- **Tách riêng phần đặc thù framework:** Framework (như Laravel) thường gộp cả xử lý HTTP, Routing, ORM… tức là vừa có phần kết nối (routing, controllers nhận request) vừa có phần truy cập DB (Eloquent). Để tách biệt, ta nên giới hạn phần phụ thuộc framework ở rìa ứng dụng. Ví dụ: Controller Laravel chỉ nên đóng vai trò chuyển tiếp dữ liệu request vào lớp Application Service (thuộc core code) và trả về response. Các model Eloquent (Active Record) nếu sử dụng, nên được bao hoặc ánh xạ sang Entity thuần của domain để tránh đưa thẳng Eloquent khắp nơi. Càng ít code "Laravel" xuất hiện trong logic domain càng tốt: khi đó nếu mai này chuyển sang framework khác, domain code vẫn có thể giữ nguyên, chỉ cần viết lớp kết nối mới. Tương tự, các Facade hay Helper toàn cục

của Laravel không nên được gọi trực tiếp trong core code; thay vào đó, nên truyền chúng vào qua interface nếu thật cần thiết.

Tựu trung, giải pháp là tạo một ranh giới rõ ràng (boundary) giữa ứng dụng của ta và thế giới bên ngoài. Thế giới bên ngoài bao gồm: framework, cơ sở dữ liệu, hệ thống tệp, dịch vụ khác, v.v. Phần code giao tiếp qua lại với ranh giới đó chính là infrastructure code – ta giữ nó tách khỏi phần logic bên trong. Ranh giới này đôi khi được ví như "Ports and Adapters": core code cung cấp các "port" (cổng kết nối - thường là các interface), còn infrastructure code là các "adapter" cắm vào cổng đó để thực hiện công việc cụ thể. Kiểu kiến trúc này sẽ cho phép chúng ta hoán đổi adapter dễ dàng, ví dụ thay storage, thay phương thức giao tiếp, mà không làm xáo trộn code lõi.

Ví dụ minh họa

Để hình dung rõ hơn, ta xem xét một ví dụ đơn giản về chức năng đặt hàng e-book trong một ứng dụng.

Giả sử ban đầu, chức năng này được hiện thực trong một phương thức controller như sau (giả định dùng Laravel, code minh họa chưa refactor):

```
public function orderEbook(Request $request)
{
    // Lấy giá ebook từ database
    $ebookId = $request->input('ebook_id');

    $ebookPrice = \DB::table('ebooks')
        ->where('id', $ebookId)
        ->value('price');

    // Tính tổng tiền đơn hàng
    $quantity = (int) $request->input('quantity');
    $orderAmount = $quantity * (int) $ebookPrice;

    // Tạo bản ghi đơn hàng và lưu vào database
```

```
    $record = [
        'email'    => $request->input('email_address'),
        'quantity' => $quantity,
        'amount'   => $orderAmount,
    ];

    $orderId = \DB::table('orders')->insertGetId($record);

    // Lưu ID đơn hàng vào session
    $request->session()->put('currentOrderId', $orderId);

    // Trả về phản hồi (view) cho người dùng
    // return view('orders.success', ['orderId' => $orderId]);
}
```

Trong đoạn code trên, ta thấy logic nghiệp vụ (tính toán số tiền, lưu thông tin đơn hàng) đang đan xen với chi tiết hạ tầng:

- Lấy giá ebook: phụ thuộc vào cơ sở dữ liệu (truy vấn bảng ebooks).
- Lưu đơn hàng: thao tác trực tiếp với bảng orders trong DB.
- Lưu ID vào session: phụ thuộc vào session của framework.

Phần tính toán `orderAmount = quantity * price` là logic nghiệp vụ thuần, nhưng nó bị kẹp giữa những đoạn truy cập DB và session. Hậu quả là muốn hiểu logic "đặt hàng e-book" bao gồm những bước gì, ta phải đọc cả đoạn mã và lọc ra khỏi đầu những chi tiết như truy vấn SQL. Code trên cũng mắc các vấn đề đã liệt kê: khó test (vì phải có DB và session), phụ thuộc chặt vào MySQL (câu SQL viết cố định), không kiểm tra gì tính hợp lệ (email có thể rỗng hoặc sai định dạng vẫn ghi vào DB, số lượng có thể là 0...). Rõ ràng đây là code “mixed” (trộn lẫn) cần được cải thiện.

Ở các chương tiếp theo, chúng ta sẽ lần lượt áp dụng những kỹ thuật refactor để tách phần core và infrastructure trong ví dụ này. Mục tiêu cuối cùng là biến đoạn code trên thành một phiên bản “sạch” hơn, trong đó phần nghiệp vụ và phần hạ tầng tách rời. Ngay sau đây, mình sẽ giới thiệu

các khái niệm và mẫu thiết kế nền tảng phục vụ cho việc refactor đó: **Domain Model và Repository**.

Định nghĩa Core Code và Infrastructure Code

Core code là gì?

Core code (code lõi) của một ứng dụng là phần code có thể chạy được trong bất kỳ context (ngữ cảnh) nào mà không cần bất kỳ hệ thống bên ngoài nào hay môi trường đặc biệt nào hỗ trợ . Nói cách khác, core code hoàn toàn độc lập với cơ sở dữ liệu, network, file system, hay framework. Nếu bạn có thể lấy một đoạn code, đưa nó vào một script PHP độc lập và chạy thành công chỉ với bộ nhớ CPU, thì đoạn code đó có khả năng là core code.

Core code được định nghĩa thông qua hai quy tắc quan trọng:

- **Quy tắc 1:** Không phụ thuộc trực tiếp vào các hệ thống bên ngoài. Core code không được gọi trực tiếp các thành phần bên ngoài ứng dụng, cũng như không phụ thuộc vào code được viết riêng để tương tác với một hệ thống ngoài cụ thể . Hệ thống bên ngoài ở đây bao gồm: cơ sở dữ liệu, các remote API, hệ thống file, system clock (đồng hồ hệ thống), thư viện gửi email SMTP, v.v. Code lõi phải có khả năng chạy tốt mà không cần những thứ đó. Điều này không có nghĩa core code không được sử dụng bất kỳ abstraction nào: nó có thể gọi các interface, nhưng miễn là interface đó không tiết lộ chi tiết về hệ thống bên ngoài cụ thể. Nếu vi phạm quy tắc này, đoạn code sẽ được xem là infrastructure code.
- **Quy tắc 2:** Không cần môi trường/hệ sinh thái đặc biệt để chạy. Core code có thể chạy trong mọi bối cảnh, không đòi hỏi chỉ chạy trong web server hay CLI hay một framework cụ thể nào . Ví dụ, một class core code không được giả định rằng nó đang chạy trong request web (không được dùng biến global của framework, không dùng $_POST trực tiếp, không phụ thuộc Session, v.v.). Tương tự, core code cũng không được phụ thuộc các extension hoặc library mà chỉ hoạt động trong môi trường cụ thể. Nếu để chạy hàm đó mà phải có Laravel khởi động container, hoặc phải có biến môi trường đặc biệt, thì hàm đó chưa phải core code thuần túy. Nếu một đoạn code tuân thủ cả hai quy tắc trên,

ta có thể xem nó là core code. Ngược lại, bất kỳ code nào không thỏa mãn các quy tắc của core code đều được xem là infrastructure code .

Ví dụ về core code

- Một hàm tính toán thuế VAT từ giá sản phẩm: nó chỉ nhận đầu vào là các con số, thực hiện phép tính và trả về kết quả. Hàm này không đọc database, không gọi API, không phụ thuộc gì ngoài chính dữ liệu truyền vào: đây là core code điển hình. Bạn có thể gọi hàm này mọi lúc mọi nơi, trong unit test hay trong ứng dụng thực tế đều được, không cần set up gì đặc biệt.
- Một phương thức của đối tượng domain, ví dụ `$order->calculateTotalAmount()` chỉ dùng các thuộc tính đã có của $order (như đơn giá, số lượng) để tính ra tổng tiền. Nó không truy cập bất kỳ service bên ngoài hay global state nào. Phương thức này có thể chạy độc lập, vì vậy logic bên trong nó thuộc về core code.
- Lớp `OrderService` (thuộc tầng ứng dụng) với phương thức placeOrder() chẳng hạn, nếu được thiết kế đúng, sẽ lấy vào các giá trị cần thiết, tạo đối tượng domain, gọi các phương thức xử lý domain (như giảm tồn kho, tính khuyến mãi) và trả về kết quả là một entity hoặc giá trị nào đó. Nếu trong quá trình đó, OrderService chỉ thao tác trên domain model và sử dụng các interface (như `PaymentGatewayInterface`, `OrderRepositoryInterface`) chứ không chạm tới database hay request, thì OrderService đó cũng có thể được coi là core code (thuộc Application layer nhưng không chứa code hạ tầng).

Điểm mấu chốt: core code có thể được thực thi hoàn toàn trong bộ nhớ, không đòi hỏi hạ tầng ngoài. Điều này làm cho core code rất dễ kiểm thử: ta có thể tạo đối tượng và gọi phương thức core code mà không cần cấu hình gì phức tạp. Khi code tuân theo quy tắc core, ta có thể chạy nó trong cô lập hoàn toàn. Viết test tự động cho core code trở nên đơn giản: không cần thiết lập database, không cần internet, không cần file đặc biệt: chỉ cần bộ nhớ và CPU để chạy code là đủ. Đây chính là lợi thế lớn về testability của core code.

Ngoài ra, core code còn có đặc trưng là di động: bạn có thể tái sử dụng core code trong bối cảnh khác nhau (web app, console app, một script nhỏ) mà không gặp trở ngại. Ví dụ, một module tính toán giá vận chuyển (shipping cost) viết thuần tuý (core) có thể dùng lại trong nhiều dự án khác nhau chỉ bằng cách import module, vì nó không kéo theo phụ thuộc hạ tầng.

Infrastructure code là gì?

Infrastructure code (code hạ tầng), ngược lại, là bất kỳ code nào không thoả mãn tiêu chí core code. Nói một cách đơn giản, infrastructure code là phần code mà muốn chạy được thì phải có cái gì đó bên ngoài hỗ trợ: có thể là kết nối tới hệ thống khác, hoặc phải chạy trong một môi trường đặc thù. Infrastructure code là loại code kết nối logic lõi của ứng dụng với các hệ thống xung quanh như cơ sở dữ liệu, web server, file system, v.v. . Nó thường bao gồm: code truy vấn và thao tác cơ sở dữ liệu, code gọi đến dịch vụ web bên ngoài (thông qua HTTP, gRPC...), code tương tác với hệ thống tập tin (đọc ghi file), code gửi email hoặc message, code ghi log ra hệ thống ngoài, và đương nhiên cả code thuộc về framework (ví dụ controller, model của framework nếu chúng gắn liền với ORM)... Một cách nhận biết: nếu bạn nhìn vào code và thấy nó chứa các chi tiết kỹ thuật về bên ngoài (chuỗi DSN kết nối DB, tên bảng, câu lệnh SQL, đường dẫn URL gọi API, tên file, query HTTP request, v.v.) thì rất có thể đó là infrastructure code. Những chi tiết này cho thấy code đang tương tác với thứ gì ngoài bản thân nó.

Ví dụ về infrastructure code

- Bất kỳ đoạn code nào thực hiện truy vấn SQL hoặc thao tác ORM. Ví dụ dùng PDO: `$pdo->query("SELECT * FROM users")` hoặc dùng Eloquent: `User::find(5)` . Những lệnh này đòi hỏi có kết nối DB thực sự mới chạy được, do đó chúng thuộc hạ tầng. Ngay cả khi được gọi thông qua một interface trừu tượng (như `ConnectionInterface` chẳng hạn), bản chất chúng phục vụ việc tương tác DB nên vẫn là infrastructure code .
- Code gọi API ngoài, ví dụ sử dụng cURL hoặc HTTP client để gửi request: `Http::get('https:// api.bank.com/rates')` . Nếu API không chạy hoặc không có mạng, code sẽ không hoạt động. Những thao tác này rõ ràng phụ thuộc hệ thống ngoài (ở đây là dịch vụ HTTP bên thứ ba), nên chúng là infrastructure.
- Đoạn code đọc ghi file, ví dụ: `file_get_contents('/path/to/config.json')` hoặc thao tác với filesystem thông qua SplFileObject, cũng như code gửi email qua thư viện PHPMailer, đều đòi hỏi có tài nguyên hệ thống (file, mail server) bên ngoài, nên đều là hạ tầng .
- Phần lớn code thuộc framework: ví dụ trong Laravel, các class Controller (mở rộng từ `App\Http\Controllers\Controller`) thực tế chỉ chạy được trong bối cảnh Laravel (vì nó dựa vào Request, Response của framework, container IoC...). Tương tự, một Eloquent

Model (kế thừa `Illuminate\Database\Eloquent\Model`) chính là một `Active Record` gắn với DB, cần kết nối DB và context Laravel để hoạt động đầy đủ, nên nó mang tính infrastructure. Đa số code trong thư mục /vendor (chứa các package, bao gồm framework) là infrastructure code, bởi chúng được thiết kế để chạy trong những ngữ cảnh đặc thù (web, console) và phụ thuộc các hệ thống như DB, HTTP... . Dĩ nhiên, cũng có một số ngoại lệ: chẳng hạn một thư viện tính toán thuần tuý nằm trong /vendor có thể là core code vì nó độc lập.

Tóm lại, infrastructure code = (tổng code) - (core code). Bất cứ chỗ nào core code dừng lại, cần "ra ngoài" để làm gì đó, thì phần "ra ngoài" đó là hạ tầng. Chẳng hạn core code quyết định "gửi email xác nhận đơn hàng" – việc thực sự gửi email thông qua SMTP hay API dịch vụ email là việc của infrastructure code. Core code quyết định "lưu đơn hàng" – việc thực thi lưu (SQL INSERT vào bảng, hay gọi repository cụ thể) là của infrastructure.

Một lưu ý quan trọng: sự trừu tượng hóa (abstraction) không tự động biến code hạ tầng thành core code. Ví dụ, bạn có thể tạo một interface Mailer và code core gọi `Mailer->send($message)` . Nếu implementation của Mailer dùng PHP mail() hoặc dùng SMTP, rõ ràng đó là hạ tầng. Interface giúp ta tách bạch phụ thuộc, nhưng bản chất chức năng gửi email vẫn là một concern hạ tầng. Do đó phần code thực thi (dù gián tiếp qua interface) vẫn thuộc infrastructure. Ví dụ `Connection interface` có phương thức `insert($table, $data)`: dù class core `UserRegistration` gọi qua interface này thay vì gọi PDO trực tiếp, nó vẫn bị coi là đang phụ thuộc một thứ dành riêng cho database (vì interface đó abstract một khái niệm thuộc về DB). Kết luận: `registerUser()` trong trường hợp này vẫn là infrastructure code chứ không phải core code, do nó "vướng" phải chi tiết về DB (bảng users). Do vậy, đừng lẫn lộn: chỉ khi abstraction đủ tách rời khỏi chi tiết kỹ thuật bên dưới thì code dùng abstraction đó mới thoát khỏi hạ tầng. Nếu interface quá sát với hạ tầng (như interface kết nối DB, interface gọi HTTP client), thì code dùng nó vẫn gián tiếp mang tính hạ tầng.

Ranh giới giữa core và infrastructure trong thực tế

Trong thực tế dự án, không phải lúc nào cũng rõ ràng 100% đoạn code nào là core hay infra: đôi khi ranh giới có thể mờ nhạt, khó nhận biết.

- Code core thường tập trung trong các lớp thuộc domain và application logic, ví dụ các class nghiệp vụ, các service hoặc use case thuần logic. Chúng có thể sử dụng các Value Object (như đối tượng Money, DateRange) để tránh phụ thuộc kiểu dữ liệu nguyên thủy

của hệ thống (VD: dùng Money thay vì float, dùng DateTimeImmutable thay vì timestamp thô). Những giá trị đối tượng này nếu tự kiểm soát logic và không gọi hệ thống ngoài thì chúng cũng là core code.

- Code infra thường nằm ở lớp ngoài cùng: các thành phần giao tiếp bên ngoài như Controller, Repository implementation, Gateway kết nối dịch vụ khác, Provider của framework, v.v. Những nơi này bạn khó tránh khỏi việc gọi đến code framework hoặc extension hệ thống, nên hãy cố gắng giới hạn chúng ở đó, không để "rò rỉ" vào domain. Ví dụ, trong Laravel Controller (infrastructure), bạn lấy dữ liệu request và gọi `PlaceOrderService` (core) xử lý; `PlaceOrderService` trả về kết quả domain (Order), controller chuyển nó thành JSON response gửi ra (hạ tầng). Như vậy chỉ controller xử lý Response/Request - domain service không biết gì về HTTP.
- Nếu vẫn phân vân, hãy áp dụng 2 quy tắc đã nêu: Không phụ thuộc hệ thống ngoài? Không cần môi trường đặc biệt? Nếu câu trả lời "có" cho cả hai, đó là core. Chỉ cần một tiêu chí "không", thì đó là infra. Ví dụ, một class `Event Dispatcher` của Laravel (giả sử ta dùng nó): nó có vẻ là thành phần hệ thống, nhưng nó có cần hệ thống ngoài không? Bản thân dispatch sự kiện nội bộ không cần DB hay network, nhưng nó bị thiết kế để hoạt động trong môi trường Laravel, có thể đòi hỏi context của framework -> vậy có thể coi là infra (vì "cần môi trường đặc biệt"). Thực tế, ta thường xem thành phần này là một phần của framework (nên là hạ tầng).
- Đừng phụ thuộc vào vị trí thư mục, hãy phụ thuộc vào chức năng: Như đã nói, không phải cứ code trong folder `App/Models` hay `App/Http` thì chắc chắn hạ tầng, cũng không phải cứ code trong `App/Services` là core. Bạn phải nhìn vào nội dung. Ví dụ: một class tính toán lãi suất trong App\Utils\Calculator dù ở sâu trong code ta viết, nhưng nếu bên trong nó lại gọi cURL tới một API lấy tỷ giá thì nó vẫn là infra (vì phụ thuộc web service). Ngược lại, một số code trong thư viện ngoài có thể là core nếu nó hoàn toàn không đụng hệ thống (vd: thư viện markdown parser - parse text thuần túy, có thể xem như core logic trong ngữ cảnh ứng dụng ta dùng).
- **Infrastructure code có thể được thu gọn ảnh hưởng nhờ adapter:** Mục tiêu cuối là để core code "không nhận ra" sự tồn tại của hạ tầng. Ví dụ, thay vì để mọi nơi trong app gọi `Carbon::now()` (Carbon là lib datetime của Laravel, phụ thuộc timezone hệ thống), ta tạo một interface Clock với method now() và một implementation SystemClock gọi Carbon. Ở core code, ta dùng Clock->now() . Như vậy core không biết Carbon là gì. Tương tự, thay vì dùng trực tiếp Eloquent model trong domain logic, ta có thể tạo một lớp

chuyển đổi Eloquent Model <-> Domain Entity, hoặc dùng repository để chuyển. Những adapter này bản chất là hạ tầng, nhưng chúng giúp cô lập hạ tầng tại một điểm duy nhất.

Sau khi đã hiểu rõ định nghĩa, hãy tự kiểm tra một vài trường hợp trong chính code của bạn: ví dụ, một đoạn code lấy thời gian hiện tại `now = new DateTimeImmutable('now')` có phải hạ tầng không? Theo quy tắc, việc lấy thời gian hệ thống có thể xem là phụ thuộc môi trường (đồng hồ hệ thống) - tuy nhiên nó rất nhẹ và có thể chấp nhận trong core code nếu truyền vào từ ngoài. Còn việc truy cập một `Event Dispatcher` của Laravel hay gọi phương thức static của Eloquent như `User::find(1)` chắc chắn là infrastructure. Luôn đặt câu hỏi: "Đoạn code này có chạy độc lập, trong memory, không side effect ngoài không?" để định danh.

Domain Model và Repository Pattern

Domain Model: Mô hình miền nghiệp vụ

Khi đã xác định rõ ranh giới giữa logic nghiệp vụ (core) và hạ tầng, bước tiếp theo là thiết kế Domain Model - mô hình miền nghiệp vụ - để hiện thực hóa phần core code một cách chặt chẽ, có cấu trúc. Domain Model là một tập hợp các khái niệm, đối tượng, quy tắc nghiệp vụ được mô hình hóa dưới dạng code, phản ánh sát nhất thực thể và quy trình trong thế giới thực của bài toán mà phần mềm giải quyết. Việc xây dựng domain model thường liên quan đến các ***Entity, Value Object, Domain Service***,... tuân theo tư tưởng của ***Domain-Driven Design***.

Tại sao cần Domain Model?

Xét lại ví dụ đặt hàng e-book ở chương trước: trong phiên bản code ban đầu (mixed code), logic nghiệp vụ nằm tản mát trong controller dưới dạng các thao tác thủ tục (tính toán tiền, tạo mảng dữ liệu, tính ID...). Cách tiếp cận này thường được gọi là Transaction Script: viết tuần tự các bước thực hiện nghiệp vụ. Nhược điểm của nó là logic không có chỗ "định cư" cố định, dẫn đến khó bảo trì và mở rộng. Domain Model khắc phục vấn đề này bằng cách đóng gói các dữ liệu và logic liên quan thành các đối tượng có trạng thái và hành vi, qua đó:

- **Thể hiện rõ các khái niệm domain:** Thay vì làm việc với các mảng dữ liệu vô danh ($record chứa email, quantity, amount như ví dụ), ta tạo một class Order đại diện cho đơn hàng. Điều này biến khái niệm "Đơn hàng" thành một "first class citizen, hay công dân định danh mức cao nhất" trong code - có danh tính, có thuộc tính và phương thức. Bất kỳ logic nào liên quan đến đơn hàng (tính tổng tiền, kiểm tra tính hợp lệ, trạng thái thanh toán…) sẽ nằm trong hoặc liên quan chặt đến class này, thay vì rải rác.
- **Đảm bảo tính nhất quán và đầy đủ của dữ liệu domain:** Khi dùng Domain Model, ta có cơ hội kiểm soát dữ liệu ngay tại lớp domain. Ví dụ, trong constructor của Order , ta có thể yêu cầu mọi thông tin cần thiết (email, số lượng, đơn giá, v.v.) đều có đủ một lúc. Nếu thiếu hoặc sai, constructor sẽ ném exception. Bằng cách đó, ta ngăn chặn việc tạo ra một Order không hợp lệ ngay từ đầu . Ở ví dụ ban đầu, đơn hàng được lưu bằng câu SQL mà không hề kiểm tra email có hợp lệ không, số lượng > 0 không - Domain Model sẽ thay ta quản lý những điều kiện bất biến (invariants) như vậy.
- **Đóng gói logic liên quan vào đúng chỗ:** Domain Model cho phép đặt các hành vi (method) ngay trong đối tượng thể hiện khái niệm. Ví dụ, tính tổng số tiền đặt hàng có thể được đưa vào phương thức `$order->calculateTotal()` hoặc thậm chí thực hiện ngay trong constructor của Order (dựa trên đơn giá * số lượng). Việc này vừa đảm bảo logic được tái sử dụng dễ dàng (bất cứ nơi nào cần tính tiền đều dùng method của `Order` thay vì lặp lại công thức), vừa giúp code dễ đọc (người ta thấy `$order->calculateTotal()` là hiểu ngay thay vì đọc phép nhân rải rác). Hơn nữa, logic đặt trong domain model sẽ hoạt động trên dữ liệu nội tại của object, do đó kiểm soát tốt hơn trạng thái bên trong. Domain Model có thể ngăn các thao tác sai thứ tự hoặc vi phạm quy tắc bằng cách chỉ cung cấp những phương thức hợp lệ.
- **Tách bạch hẳn domain logic khỏi các chi tiết kỹ thuật:** Một khi ta đã có các object domain "sống" trong bộ nhớ (ví dụ một đối tượng Order hoàn chỉnh), phần xử lý nghiệp vụ có thể diễn ra thông qua các tương tác giữa các object đó – thuần túy bằng code PHP – mà không cần dính đến database hay framework. Domain Model thường phối hợp với Repository pattern (sẽ nói ngay sau) để lưu trữ và tái tạo trạng thái, còn trong quá trình xử lý, domain model hoạt động như các object thông minh. Nhờ vậy core code chạy độc lập và test độc lập dễ dàng.

Thiết kế một Domain Entity

Tiếp tục với ví dụ "Đơn hàng e-book", ta sẽ thiết kế một Entity Order. Entity là loại đối tượng có đặc điểm tồn tại lâu dài (có định danh duy nhất, ví dụ Order ID, để phân biệt các instance) và thường tương ứng với các thực thể thật trong domain (đơn hàng thật, người dùng thật, sản phẩm thật...). Một Order entity sẽ có các thuộc tính như mã ebook, email khách hàng, số lượng, đơn giá, tổng tiền, v.v. Hãy hình dung khi một khách hàng đặt mua e-book, ta cần ghi nhớ những thông tin gì cho Order đó để sau này xử lý (ví dụ xử lý thanh toán, giao hàng qua email):

- ID của e-book được đặt mua.
- Email của người mua.
- Số lượng mua.
- Đơn giá tại thời điểm mua (giá mỗi e-book).
- Tổng số tiền của đơn hàng (bằng đơn giá * số lượng, có thể cần lưu để tránh thay đổi nếu giá sản phẩm đổi sau này).
- (Có thể thêm: ngày đặt, trạng thái thanh toán, mã khuyến mãi, v.v. tùy nghiệp vụ, nhưng ở ví dụ đơn giản ta xét những cái chính trên).

Trong mô hình hướng đối tượng, cách tiếp cận tốt là **cung cấp đầy đủ các dữ liệu bắt buộc đó ngay tại thời điểm tạo đối tượng**. Ta thiết kế constructor của Order yêu cầu truyền vào toàn bộ thông tin cần thiết. Nếu thiếu bất kỳ thông tin nào, sẽ không tạo được đối tượng Order. Điều này đảm bảo một Order đã được tạo thì luôn có đủ dữ liệu căn bản - không có chuyện Order "nửa vời" không có email hay chưa tính tổng tiền. Đoạn code minh họa có thể như sau:

```
final class Order
{
    private int $ebookId;
    private string $emailAddress;
    private int $quantity;
    private int $pricePerUnit;
    private int $totalAmount;

    public function __construct(
        int $ebookId,
        string $emailAddress,
```

```
        int $quantity,
        int $pricePerUnit
    ) {
        // Kiểm tra và gán giá trị
        if ($ebookId <= 0) {
            throw new \InvalidArgumentException('ID ebook phải > 0');
        }

        if (!filter_var($emailAddress, FILTER_VALIDATE_EMAIL)) {
            throw new \InvalidArgumentException('Email không hợp lệ');
        }

        if ($quantity <= 0) {
            throw new \InvalidArgumentException('Số lượng phải > 0');
        }

        if ($pricePerUnit < 0) {
            throw new \InvalidArgumentException('Đơn giá không hợp lệ');
        }

        $this->ebookId = $ebookId;
        $this->emailAddress = $emailAddress;
        $this->quantity = $quantity;
        $this->pricePerUnit = $pricePerUnit;
        $this->totalAmount = $quantity * $pricePerUnit;
    }

    // Các phương thức getter cho thuộc tính
    public function getEbookId(): int
    {
        return $this->ebookId;
    }
```

```
    public function getEmailAddress(): string
    {
        return $this->emailAddress;
    }

    public function getQuantity(): int
    {
        return $this->quantity;
    }

    public function getPricePerUnit(): int
    {
        return $this->pricePerUnit;
    }

    public function getTotalAmount(): int
    {
        return $this->totalAmount;
    }

    // (Có thể thêm các logic khác, ví dụ tính thuế, áp mã giảm giá, v.v.)
}
```

Ở ví dụ trên, `Order` được thiết kế để đảm bảo bất biến (invariant): ID phải dương, email đúng định dạng, số lượng dương, giá không âm. Nếu vi phạm, ta ném ngoại lệ ngay. Với constructor như vậy, ta đạt được:

- Order luôn có đủ thông tin (không ai tạo Order mà thiếu email hay quên tính tổng tiền, vì constructor bắt truyền đủ tham số).
- Mọi Order tạo ra đều hợp lệ ở mức cơ bản. Sẽ không có trường hợp Order có số lượng = 0 hoặc email sai định dạng lọt vào hệ thống, bởi nó đã bị chặn ở cửa tạo đối tượng.

Ngoài ra, ta tính luôn `totalAmount` trong constructor (thay vì truyền từ ngoài vào như ví dụ ban đầu). Việc này giảm rủi ro sai sót (chẳng hạn người lập trình có thể tính sai khi truyền vào). Domain Model có thể tự lo những logic "hiển nhiên" như vậy, thay vì trông chờ bên ngoài truyền vào.

Entity vs. Value Object

Trong domain model, ta cũng nên phân biệt giữa **Entity** và **Value Object**. Entity có danh tính phân biệt (Order khác Order dựa trên ID đơn hàng chẳng hạn), và thường thay đổi trạng thái theo thời gian (vd: Order ban đầu trạng thái "Pending", sau thành "Paid"...). Value Object thì không có danh tính riêng, nó đại diện cho một giá trị và thường bất biến. Ví dụ, một `EmailAddress` class có thể là Value Object: nó chỉ chứa địa chỉ email, hai EmailAddress với cùng chuỗi email được coi là bằng nhau, và nó bất biến (email đã tạo thì không đổi nội dung). Trong thiết kế nâng cao, ta có thể tạo `EmailAddress` value object để bao bọc chuỗi email, giúp validate format bên trong nó. Ở đây, để đơn giản, ta dùng string và validate trong constructor Order cũng được. Tương tự, Money hay Price có thể là value object để giữ giá tiền + currency, đảm bảo tính toán tiền đúng (tránh nhầm lẫn đơn vị). Việc sử dụng entity và value object đúng chỗ cũng giúp tăng tính biểu đạt và đúng đắn của domain model.

Domain Model hoạt động như thế nào?

Sau khi có các entity, ta có thể viết logic nghiệp vụ bằng cách tương tác giữa các entity, hoặc giữa entity với domain service. Domain service là các lớp chứa logic không gắn riêng một entity nào, nhưng thuộc domain (ví dụ dịch vụ tính phí vận chuyển dựa trên nhiều yếu tố). Ở ví dụ đặt hàng, có lẽ chưa cần domain service, ta có thể để logic trong entity hoặc application service.

Quan trọng là: domain model giúp ta chuyển từ tư duy "làm gì trước, làm gì sau" của lập trình thủ tục sang tư duy "đối tượng này làm những nhiệm vụ gì". Trình tự thực hiện nghiệp vụ sẽ được sắp xếp lại một chút. Trong trường hợp Order e-book:

- Trước đây: Controller lấy giá -> tính tiền -> tạo mảng -> lưu DB -> lấy ID -> lưu session.
- Với domain model: Controller sẽ tạo một Order entity (yêu cầu đầu vào: ebookId, email, quantity, price – những thứ này controller lấy từ request hoặc từ DB).

Khi tạo Order, nếu dữ liệu bất thường sẽ có exception (lúc đó controller có thể bắt và xử lý, ví dụ trả lỗi "Dữ liệu không hợp lệ"). Nếu tạo thành công, ta có một đối tượng Order đã tự tính toán

tổng tiền và đảm bảo hợp lệ. Sau đó, controller gọi repository để lưu Order này (thay vì tự viết câu SQL). Cuối cùng, lấy ID (do repository trả về hoặc Order mang) lưu session.
Kết quả, phần xử lý domain (tính toán, kiểm tra) đều do Order đảm nhận. Controller chỉ lo việc lấy dữ liệu đầu vào, gọi tạo Order, rồi nhờ repository lưu. Nhìn vào controller bây giờ sẽ rõ ràng hơn rất nhiều: "Tạo Order mới và lưu nó" - không còn chi tiết lưu thế nào, SQL gì, v.v. Domain model đã đẩy chi tiết đó đi.
Chúng ta sẽ thấy phần code minh họa "sau khi refactor" ở phần sau. Trước tiên, hãy cùng tìm hiểu **Repository pattern**, mảnh ghép quan trọng giúp Domain Model "làm việc" với hạ tầng dữ liệu.

Repository Pattern - Mẫu thiết kế Repository

Khi dùng domain model, ta có các đối tượng entity sống trong bộ nhớ. Nhưng làm thế nào để lưu trữ chúng vào database, và load lại sau này? Đây là lúc Repository pattern phát huy tác dụng. Repository (kho lưu trữ) là một mẫu thiết kế tạo một tầng trung gian giữa domain và data source, cung cấp các phương thức như một bộ sưu tập đối tượng để quản lý các entity. Martin Fowler định nghĩa, Repository đóng vai trò như một tập hợp hướng đối tượng cho phép tách biệt và một chiều hóa phụ thuộc giữa tầng domain và tầng mapping dữ liệu . Nói đơn giản, Repository giống như một cái kho trong đó ta có thể thêm, lấy, xóa, tìm kiếm các đối tượng domain mà không cần biết chúng được lưu ở đâu, như thế nào. Với domain code, repository được xem như một interface (cổng) để truy xuất đối tượng. Còn với infrastructure code, repository là chỗ để cài đặt chi tiết (sử dụng ORM, SQL, file, v.v.).

Định nghĩa Repository

Một repository thường được định nghĩa cho mỗi loại Entity gốc cần lưu. Ví dụ, ta có `OrderRepository` quản lý các Order, UserRepository quản lý User, v.v. Interface của repository sẽ khai báo các phương thức cần thiết, phổ biến gồm:

- `save(Entity $entity)` : lưu một entity (mới hoặc đã sửa đổi) vào nguồn lưu trữ. - `getById($id): Entity` : tìm và trả về entity với định danh $id (có thể ném exception nếu không tìm thấy).
- Có thể có các phương thức truy vấn khác tùy nhu cầu: `findByUser($userId)` , `findAllPendingOrders()` ,… hoặc phân tích hơn thì có repository cho tập hợp (collection) nhưng ta không đi sâu mô hình DDD phức tạp ở đây.

- Đối với thao tác xóa: có thể có remove(Entity $entity) hoặc thực hiện ngầm khi entity out of scope tùy pattern, nhưng thường có method delete.

Repository pattern nhấn mạnh rằng domain code chỉ tương tác với repository qua interface. Domain code không biết chi tiết repository làm việc thế nào. Điều này vừa giúp tách biệt hạ tầng (vì implement nằm ở hạ tầng), vừa cho phép thay đổi chiến lược lưu trữ dễ dàng. Ta có thể có `SqlOrderRepository` (lưu vào MySQL), `MongoOrderRepository` (lưu MongoDB), thậm chí `MemoryOrderRepository` (lưu vào mảng, dùng trong test): tất cả đều triển khai OrderRepository interface. Khi chạy ứng dụng thật, ta bind interface `OrderRepository` với `SqlOrderRepository` ; khi test, ta bind với `MemoryOrderRepository` nếu muốn, chẳng hạn. Domain service hay controller chỉ biết gọi `OrderRepository->save($order)` , còn việc `$order` đi đâu, được lưu ra sao thì tùy phiên bản cụ thể. Repository như giải pháp cho vấn đề "cần lưu trữ một đối tượng domain và sau đó tái tạo nó".

Ban đầu, ta có thể "tưởng tượng" ra repository ngay cả khi chưa có nó: giả sử trong code nếu có `$orderRepository->save($order)` thì tiện quá: vậy hãy định nghĩa interface `OrderRepository` với method `save(Order $order)` đúng như mong muốn . Đây là một mẹo thiết kế: ***giả định có sẵn một thành phần mình cần rồi định nghĩa nó, sau đó mới lo cài đặt***.

Ví dụ thiết kế OrderRepository

Tiếp tục bài toán Order e-book, ta tạo interface `OrderRepository` với phương thức `save` nhận một Order. Vì trong DB, Order ID (giả sử auto-increment) được tạo khi insert, nên có thể thiết kế save() trả về ID (kiểu int) của Order vừa lưu . Ngoài ra thường repository có `getById($orderId): Order` . Nhưng trong bối cảnh refactor nhỏ (chỉ lo lưu một đơn hàng trong luồng đặt hàng), ta có thể tạm thời chưa cần `getById` . Dù sao, ta biết về nguyên tắc repository thường có cặp `save` và `getById` bổ sung cho nhau.

```
interface OrderRepository
{
    /**
     * Lưu Order, trả về ID vừa lưu
     */
    public function save(Order $order): int;

    /**
```

```
     * Tìm Order theo ID, ném ngoại lệ nếu không tìm thấy
     */
    public function getById(int $orderId): Order;
}
```

(Ghi chú: Nếu dùng UUID tự sinh cho Order thì có thể không cần trả về int ID. Nhưng ở đây giả định dùng autoincrement ID do DB cấp.)

Trong domain layer, OrderRepository là một hợp đồng (contract) thể hiện khả năng lưu và truy xuất Order. Domain code (vd. Application service thực thi use case đặt hàng) sẽ chỉ quan tâm đến contract này.

Triển khai Repository ở tầng hạ tầng

Bây giờ, ở tầng Infrastructure, ta viết lớp cụ thể, ví dụ `SqlOrderRepository` cài đặt `OrderRepository`. Lớp này có nhiệm vụ chuyển đổi qua lại giữa đối tượng Order và biểu diễn để lưu (chẳng hạn mảng dữ liệu, hoặc query builder). Sử dụng PDO hoặc Query Builder để insert vào DB. Với Laravel, ta có thể chọn dùng Query Builder (`DB::table`) hoặc dùng Eloquent. Mỗi cách có ưu nhược:

- Dùng Query Builder/DB: ta thao tác SQL thủ công, nhưng rõ ràng và không phụ thuộc Active Record của Eloquent. Nó phù hợp tinh thần Data Mapper (mapping thủ công).
- Dùng Eloquent (Active Record): ta có thể tận dụng model Eloquent để lưu. Song Eloquent Model bản thân nó là Active Record, nếu truyền thẳng entity domain vào Eloquent sẽ phức tạp (cần convert). Có một cách là tạo một Eloquent model ẩn bên trong repository và dùng nó để lưu. Dù sao, Eloquent vẫn là hạ tầng nên ta bao trong repository cũng được.

Ở đây, ta thử triển khai `SqlOrderRepository` dùng Query Builder của Laravel cho đơn giản:

```
use App\Domain\Order; // Giả sử Order nằm trong App\Domain
use App\Domain\OrderRepository;
use Illuminate\Support\Facades\DB;
```

```php
final class SqlOrderRepository implements OrderRepository
{
    public function save(Order $order): int
    {
        // Chuyển đổi Order entity thành dữ liệu để lưu
        $data = [
            'ebook_id' => $order->getEbookId(),
            'email'    => $order->getEmailAddress(),
            'quantity' => $order->getQuantity(),
            'amount'   => $order->getTotalAmount(),
        ];

        // Thực hiện insert vào database
        $orderId = DB::table('orders')->insertGetId($data);

        return $orderId;
    }

    public function getById(int $orderId): Order
    {
        $record = DB::table('orders')
            ->where('id', $orderId)
            ->first();

        if (!$record) {
            throw new \RuntimeException('Order not found');
        }

        // Tạo lại Order entity từ dữ liệu DB
        return new Order(
            $record->ebook_id,
```

```
            $record->email,
            $record->quantity,
              (int) ($record->amount / $record->quantity) // pricePerUnit =
amount / quantity
        );
    }
}
```

Trên đây, `SqlOrderRepository` thực hiện việc cần làm:

- Trong `save()`: nhận một Order, lấy dữ liệu cần (ebookId, email, quantity, amount) và chèn vào bảng orders . Laravel cung cấp hàm insertGetId để chèn và lấy ID. Sau đó trả về ID đó (để caller dùng nếu cần).
- Trong `getById()` : chạy query lấy bản ghi theo ID. Nếu không thấy, ném ngoại lệ. Nếu có, khởi tạo một đối tượng Order từ dữ liệu. Lưu ý: khi tạo Order, ta cần truyền đủ tham số: ebookId, email, quantity, pricePerUnit. Ở DB ta có amount tổng tiền và 18 quantity , có thể suy ra pricePerUnit . Ở đây làm phép chia (cần cẩn thận số nguyên). Thực ra ví dụ này đơn giản, trong thực tế ta thường lưu hẳn `price_per_unit` ở cột riêng để không phải chia như trên.

Lưu ý: Với Eloquent, ta có thể cài đặt khác: Ví dụ, `EloquentOrderRepository` có thể được minh họa bằng cách gọi `$order->save()` nếu Order là một Active Record entity . Nhưng trong thiết kế của ta, Order là một entity thuần, không kế thừa Eloquent. Một cách là dùng Eloquent Model ẩn: tạo model Eloquent `OrderModel` trỏ bảng orders , rồi trong repository save() thì:

```
$orderModel = new OrderModel();
$orderModel->fill([...]);
$orderModel->save();
return $orderModel->id;
```

và `getById()` thì `OrderModel::findOrFail($id)` rồi map sang domain Order. Cách này cũng tương tự việc dùng Query Builder, chỉ khác là tận dụng ORM để map cho mình. Tùy trường hợp mà ta chọn cách triển khai.

Điều quan trọng: Repository implementation là chi tiết hạ tầng. Nó biết rõ ứng dụng dùng DB loại gì, bảng gì, thậm chí câu SQL cụ thể. Những thông tin này không hề lộ ra domain - domain

chỉ biết có phương thức `save()` trừu tượng. Bởi vậy, nếu ngày mai ta đổi lưu Order sang một dịch vụ web khác (ví dụ gọi API của một service để lưu đơn hàng), ta chỉ cần viết một implementation `RemoteOrderRepository` và bind thay thế, domain code không đổi gì. **Khả năng thay thế này chính là sức mạnh decoupling.**

Sử dụng Domain Model + Repository trong luồng nghiệp vụ

Bây giờ hãy xem lại chức năng đặt hàng e-book sau khi có Domain Model và Repository.
Giả sử ta có `OrderController` (thuộc hạ tầng web) xử lý request đặt hàng. Controller này sẽ không làm tất cả như ban đầu nữa, mà ủy thác cho domain logic. Cụ thể:

- Controller nhận Request $request với các tham số.
- Controller lấy các tham số cần thiết: ebookId, email, quantity.
- Controller có OrderRepository (được inject qua constructor chẳng hạn, nhờ IoC container).
- Controller có thể cần biết giá ebook. Đây là một chi tiết: giá là dữ liệu từ DB (bảng ebooks). Ta xử lý nó ở đâu? Có nhiều cách: **Cách A:** Tra giá ở controller bằng một repository khác hoặc query rồi truyền vào Order. **Cách B:** Sử dụng một Domain Service để tạo Order, trong đó domain service sẽ dùng repository để lấy giá. Nhưng nếu domain service làm vậy sẽ lại dính hạ tầng (vì lấy giá từ DB) - trừ phi ta cũng trừu tượng hóa nguồn giá. **Cách C:** Truyền hẳn repository của ebook (hoặc một `PricingService`) vào Order như một dependency - phức tạp quá cho ví dụ nhỏ. Ở đây, ta làm đơn giản: Controller tạm thời vẫn truy vấn giá (sử dụng một `EbookRepository` tương tự OrderRepository để get price) rồi truyền giá đó khi tạo Order. Điều này hơi "nửa vời" vì controller vẫn dính DB, nhưng có thể cải tiến bằng cách có một Application Service lo việc này. Để không làm loãng, cứ cho controller lấy giá, ta sẽ refactor sau (Application Service có thể tách khỏi controller).

Vậy ta có thể refactor theo các bước:

- Controller tạo đối tượng Order: `$order = new Order($ebookId, $email, $quantity, $price);` (có thể phải catch exception nếu dữ liệu xấu).
- Gọi repository: `$orderId = $this->orderRepository->save($order);` Lưu `$orderId` vào session (hạ tầng).
- Trả Response.

Code ví dụ:

```php
class OrderController extends Controller
{
    private OrderRepository $orderRepo;
    private EbookRepository $ebookRepo;

     public function __construct(OrderRepository $orderRepo, EbookRepository
$ebookRepo)
    {
        $this->orderRepo = $orderRepo;
        $this->ebookRepo = $ebookRepo;
    }

    public function orderEbook(Request $request): Response
    {
        $ebookId = (int) $request->input('ebook_id');
        $quantity = (int) $request->input('quantity');
        $email = $request->input('email_address');

        // Lấy giá e-book từ repository (hạ tầng truy vấn DB)
        $price = $this->ebookRepo->getPriceById($ebookId);

        try {
            // Tạo Order entity (domain)
            $order = new Order($ebookId, $email, $quantity, $price);
        } catch (\Exception $e) {
            // Dữ liệu không hợp lệ → trả về lỗi cho người dùng
            return response(
                'Thông tin đơn hàng không hợp lệ: ' . $e->getMessage(),
                400
            );
        }
```

```
        // Lưu Order thông qua repository
        $orderId = $this->orderRepo->save($order);

        // Lưu ID đơn hàng vào session (hạ tầng)
        $request->session()->put('currentOrderId', $orderId);

         // Trả về response (ví dụ redirect tới trang hiển thị đơn hàng vừa đặt)
                return  redirect()->route('order.success',  ['order_id'  => $orderId]);
    }
}
```

Nhìn vào controller sau refactor, ta thấy rõ sự khác biệt:

- Phần logic domain (tính toán tổng tiền, kiểm tra dữ liệu) đã không còn trong controller; nó nằm kín đáo bên trong Order entity.
- Controller vẫn còn chút hạ tầng là lấy giá e-book. Ta có thể tưởng tượng có `EbookRepository` làm việc này (như code trên). Nếu không, controller dùng trực tiếp DB::... cũng là hạ tầng. Dù sao, việc đó cũng có thể đẩy vào domain nếu ta thiết kế domain service. Nhưng để đơn giản, chấp nhận controller làm việc này.
- Controller gọi `$orderRepo->save($order)` : chi tiết lưu thế nào (SQL) ẩn sau repository.
- Code controller trở nên ngắn gọn, dễ hiểu: nhận request -> tạo đối tượng domain -> lưu qua repository -> trả kết quả.

Về tính đúng đắn và clean:

- Ta không còn lo quên tính `totalAmount` hay để sai email - vì Order entity lo rồi.
- Dữ liệu vào DB đảm bảo đã được validate cơ bản. - Nếu sau này cần thay đổi cách lưu (vd lưu sang hệ thống khác), chỉ chỉnh `OrderRepository` binding, controller giữ nguyên.
- Thử tưởng tượng test controller: ta có thể mock `EbookRepository` cho trả về giá cố định, mock `OrderRepository` để kiểm tra nó có nhận được Order đúng không – và không cần DB thật khi test controller. Việc test Order ta làm riêng (unit test Order), test `SqlOrderRepository` có thể làm integration test riêng với DB. Mỗi thành phần tách biệt test được, so với ban đầu muốn test controller gốc cần DB thật.

Phân tích lợi ích đạt được

Kết quả refactor sang Domain Model + Repository đã đem lại:

- **Core code gói trong Order entity và `OrderRepository` interface:** cả hai đều không phụ thuộc hạ tầng cụ thể. Order entity hoàn toàn độc lập (no external dependency). `OrderRepository` interface cũng độc lập (chỉ là contract).
- **Infrastructure code cô lập trong `SqlOrderRepository`:** Mọi thứ liên quan MySQL, bảng orders chỉ nằm trong lớp này. Nếu cần thay đổi schema, ta biết chỉ đụng đến đây. Core code không biết về tên bảng, cột: "mù tịt" về SQL.
- **Code trong controller nhẹ nhàng, đóng vai trò Orchestrator - điều phối dữ liệu giữa các thành phần**. Ta có thể coi controller là một phần của Application layer, sử dụng domain và repository.

Sau khi trích xuất Entity và Repository, ta có thể đánh dấu code này là core code, bởi: Order entity và OrderRepository interface không để lộ bất kỳ chi tiết nào về client hay môi trường - chúng thuần về nghiệp vụ . Thật vậy, Order chỉ chứa dữ liệu và logic nội tại; OrderRepository interface chỉ nói lên ý nghĩa "lưu Order" chứ không nhắc đến MySQL hay file hay gì khác. OrderRepository interface bản thân nó là một abstraction, cho phép ta chạy các phần core khác mà không cần database thực sự hoặc phụ thuộc ngoài nào - ta có thể cung cấp implementation giả để chạy domain logic trong isolation . Order entity có thể được dùng mà không cần web server, DB hay bất cứ hạ tầng nào: nó chỉ là một lớp PHP thuần (POPO – Plain Old PHP Object), tạo và dùng không cần setup đặc biệt . Nói cách khác, sau refactor, phần cốt lõi của chức năng "đặt hàng" đã trở thành core code thực sự. Chúng ta đã đạt được mục tiêu tách biệt: phần core (entity + interface) độc lập, phần hạ tầng (repository implementation, controller) kết nối qua abstraction.

Read Models và View Models

Vấn đề khi dùng Entity cho mục đích đọc: Một sai lầm phổ biến là tái sử dụng entity của write model cho mục đích đọc dữ liệu (hiển thị, báo cáo). Thoạt nhìn, việc dùng lại các đối tượng domain sẵn có để lấy thông tin có vẻ tiện lợi, nhưng nó gây nhiều hệ lụy. Trước hết, các entity được thiết kế để xử lý nghiệp vụ (ghi dữ liệu) thường có trạng thái biến đổi được và phương thức thay đổi trạng thái (ví dụ Ebook- >changePrice() , Ebook->hide() trong domain e-book) . Khi ta

load một entity chỉ để đọc thông tin (ví dụ lấy giá e-book), entity đó vẫn mang theo hàng loạt phương thức cho phép thay đổi nó - trao cho phần code trình bày (vốn chỉ cần dữ liệu để hiển thị) quyền can thiệp không cần thiết. Điều này vi phạm nguyên tắc chỉ cung cấp những gì cần thiết: "Chỉ nên cho client những phương thức mà nó thực sự cần". Nếu tầng hiển thị vô tình gọi nhầm phương thức thay đổi trạng thái hoặc làm rối logic domain, sẽ dẫn đến lỗi khó lường.

Thứ hai, **tái sử dụng một model cho nhiều mục đích khiến model phình to và khó bảo trì**. Khi một class vừa dùng để ghi (write) vừa dùng để đọc, nó phải đảm đương hai vai trò, dẫn đến việc thêm nhiều thuộc tính/phương thức phục vụ cả hai nhu cầu. Dần dần class sẽ quá lớn và "đa năng", việc chỉnh sửa trở nên khó khăn vì sợ ảnh hưởng đến ngữ cảnh khác đang dùng nó . Các phương thức phục vụ mục đích đọc có thể mâu thuẫn hoặc trùng lặp với logic ghi, và ngược lại. Về lâu dài, đối tượng như vậy trở thành code legacy, khó thay đổi do phụ thuộc ngầm chằng chịt và phải chiều nhiều client khác nhau . Do đó, một nguyên tắc tổ chức hữu ích là tách biệt mô hình ghi (write model) và mô hình đọc (read model). Phần cần thay đổi trạng thái (ghi) nên dùng các entity/aggregate thiết kế cho nghiệp vụ, còn phần chỉ cần dữ liệu để hiển thị (đọc) thì dùng mô hình khác nhẹ hơn, tối ưu cho việc truy vấn.

Triển khai Read Model: Read Model là mô hình dữ liệu chuyên phục vụ cho các thao tác đọc/tra cứu thông tin. Thay vì trả về entity domain "đủ thứ" rồi chỉ dùng một ít, read model cung cấp đúng và chỉ những dữ liệu cần thiết cho màn hình hoặc báo cáo cụ thể. Có nhiều cách triển khai read model. Một cách là định nghĩa các class và repository riêng cho read model tương ứng với mỗi entity trong write model (nhưng ở namespace khác để tránh nhầm lẫn). Ví dụ, ta tạo một class EbookReadModel với thuộc tính chỉ gồm những thông tin cần hiển thị (vd. id , title , price), và EbookReadRepository chỉ có phương thức truy vấn phục vụ đọc (vd. findById($id) trả về EbookReadModel) . Trong trường hợp chỉ cần một giá trị đơn, ta thậm chí không cần class read model riêng mà chỉ cần một phương thức ở repository trả về kiểu nguyên thủy. Ví dụ: thay vì lấy cả đối tượng e-book, có thể có interface GetPrice với hàm ofEbook(EbookId): int trả về giá tiền trực tiếp.

Một lựa chọn khác cho read model (thường gặp trong kiến trúc CQRS) là **duy trì cơ sở dữ liệu hoặc cấu trúc lưu trữ dành riêng cho việc đọc, được cập nhật mỗi khi write model thay đổi**. Ví dụ, ta có thể có bảng hoặc view tổng hợp để phục vụ truy vấn phức tạp (như báo cáo doanh thu, danh sách đơn hàng), thay vì join nhiều bảng mỗi lần. Việc đồng bộ có thể thực hiện qua domain

event: khi entity Order được tạo hoặc cập nhật, phát ra sự kiện, một event handler sẽ cập nhật dữ liệu tương ứng trong mô hình đọc (có thể là bảng khác hoặc index tìm kiếm). Cách này giúp tách biệt hoàn toàn đọc và ghi, tối ưu hóa hiệu năng cho đọc mà không ảnh hưởng giao dịch ghi. Tuy nhiên, nó phức tạp hơn vì phải duy trì tính nhất quán giữa hai mô hình.

Trong phạm vi ebook, ta tập trung cách đơn giản: **dùng chung database nhưng lớp repository riêng cho read model**. Repository của read model có thể truy vấn trực tiếp bảng của write model. Chẳng hạn, `SqlEbookReadRepository` kết nối DB, chạy câu `SQL SELECT price FROM ebooks WHERE id = ?` để lấy giá ebook và trả về `EbookReadModel` tương ứng . Cách này dễ làm nhưng cần cẩn trọng về cách diễn giải dữ liệu: nếu logic tính toán giá thay đổi (vd chuyển từ integer cents sang decimal), repository đọc phải được cập nhật tương ứng để không hiểu sai dữ liệu . Việc chia sẻ cùng nguồn dữ liệu cũng có rủi ro "leak" logic giữa hai mô hình, nên cần có kiểm thử tích hợp đảm bảo read model khớp với write model.

View Model - mô hình hiển thị: View Model thực chất là một kiểu đặc biệt của read model, thường được dùng để đóng gói dữ liệu sẵn sàng cho View/UI hoặc API. View model có thể coi là DTO (Data Transfer Object) chứa đúng định dạng và kiểu dữ liệu mà giao diện cần, giúp việc render thuận tiện. Ví dụ, trong ứng dụng web hiển thị trang e-book, ta tạo EbookViewModel với các thuộc tính public hoặc getter như title , formattedPrice , authorName … (tất cả kiểu string) để dễ đưa vào template HTML. Khi controller nhận yêu cầu hiển thị danh sách e-book, thay vì load đối tượng domain Ebook rồi trích thông tin, nó gọi EbookViewRepository->listAll() để nhận về danh sách EbookViewModel đã có sẵn dữ liệu dạng chuỗi (price đã định dạng tiền tệ, v.v.) . Điều này giảm thiểu logic định dạng trong view và tách bạch việc chuyển đổi dữ liệu ra khỏi controller.

Ví dụ: API danh sách đơn hàng dùng ViewModel JSON: Giả sử ta có một API trả về danh sách đơn hàng mới nhất dưới dạng JSON. Thay vì dùng trực tiếp Order entity (có thể chứa nhiều dữ liệu không cần thiết và phương thức business), ta thiết kế một view model cho API này, chẳng hạn OrderListViewModel . Class này chứa các field như orderId , orderDate , customerName , totalAmount ... chỉ những thông tin cần hiển thị. Tất cả đều ở dạng nguyên thủy (string, number) để tiện serialize. Ta tạo OrderListViewRepository với phương thức findRecentOrders(int $limit): OrderListViewModel[] . Implement có thể chạy một JOIN phức tạp lấy dữ liệu từ nhiều bảng (orders, customers, payments...) nhưng gói kết quả vào OrderListViewModel đơn giản.

Controller khi xử lý request /api/orders?recent=... chỉ việc gọi repository này, nhận mảng view model và trả JSON. Mỗi OrderListViewModel có thể tự cung cấp method toArray() hoặc implement JsonSerializable để dễ dàng chuyển thành mảng . Kết quả JSON trả về chứa đúng các trường API yêu cầu, không thừa không thiếu, và hoàn toàn tách khỏi cấu trúc nội tại của domain (ví dụ domain có thể có field nhạy cảm như internal notes, nhưng view model bỏ qua hoặc ẩn đi).

Tóm lại

Việc tách Read Model/View Model khỏi Write Model giúp giảm thiểu coupling giữa phần hiển thị và phần nghiệp vụ, tăng hiệu năng truy vấn và giữ cho domain model gọn gàng. Write model (entity/ aggregate) tập trung cho nghiệp vụ và đảm bảo invariants, còn read/view model tối ưu cho truy xuất dữ liệu phục vụ người dùng. Nguyên tắc là: Không dùng đối tượng thay đổi được (mutable) với logic phức tạp cho mục đích chỉ đọc – thay vào đó tạo đối tượng đơn giản, bất biến đại diện thông tin cần cung cấp. Cách thiết kế này cũng mở đường cho kiến trúc CQRS (Command Query Responsibility Segregation), nơi command (ghi) và query (đọc) tách hẳn, giúp hệ thống dễ mở rộng và bảo trì hơn.

Application Services và Command Pattern

Tách logic use case khỏi Controller

Trong các ứng dụng web truyền thống (đặc biệt với framework như Laravel, Symfony), controller thường vừa nhận request, vừa xử lý nghiệp vụ, gọi model, lưu DB rồi trả response. Cách làm này dễ dẫn đến Controller phình to: chứa nhiều logic use case (trường hợp sử dụng cụ thể) trộn lẫn với mã xử lý HTTP. Để cải thiện, ta áp dụng nguyên tắc Separation of Concerns bằng cách đưa logic thực hiện use case ra khỏi controller, gói vào Application Service (dịch vụ ứng dụng) riêng. Application Service là lớp (hoặc component) thuộc tầng Application, chịu trách nhiệm điều phối nghiệp vụ theo một trường hợp sử dụng cụ thể, không chứa logic hạ tầng và có thể được gọi từ nhiều “entry point” khác nhau (web, CLI, test).

Ví dụ, trường hợp sử dụng “Place Order” (đặt hàng) bao gồm các bước: xác thực thông tin đầu vào, kiểm tra hàng tồn, tạo đối tượng Order, lưu Order qua repository, gửi email xác nhận, v.v. Nếu để toàn bộ trong một phương thức controller `OrderController@placeOrder` , method này sẽ rất dài và chỉ dùng được qua giao thức HTTP. Bằng cách trích xuất ra `PlaceOrderService` (một application service), ta được một lớp có thể gọi từ bất kỳ đâu (một job queue, một lệnh artisan, v.v.) mà không phụ thuộc vào đối tượng Request hay Session của web . Controller lúc này mỏng đi: nhận request, tạo DTO đầu vào, gọi service, xử lý phản hồi. Mọi logic phức tạp nằm trong service nên dễ kiểm thử (có thể unit test `PlaceOrderService` độc lập mà không cần HTTP request).

Sử dụng Command/DTO gom input

Để truyền dữ liệu đầu vào cho application service một cách rõ ràng, chúng ta dùng **Command Pattern** (ở đây hiểu là Command Object hoặc DTO - Data Transfer Object). Thay vì truyền hàng loạt tham số rời rạc vào phương thức service, ta tạo một class đơn giản biểu thị yêu cầu người dùng muốn thực hiện, chứa các thuộc tính cần thiết. Chẳng hạn, cho use case “đặt hàng”, ta định nghĩa PlaceOrderCommand với các thuộc tính: customerId , cartItems , shippingAddress , v.v. Đây là đóng gói tất cả input của use case thành một đối tượng duy nhất, giúp code gọn gàng và truyền tải ý định rõ ràng . Tên class nên diễn đạt hành động nghiệp vụ (“PlaceOrder”/“CreateOrder”), không nhầm với command-line hay hệ điều hành gì – nó đơn thuần đại diện cho ý định “người dùng muốn tạo một đơn hàng mới”.

Ví dụ, `PlaceOrderCommand` có các thuộc tính customerId, items, email , v.v. và tương ứng các getter. Constructor của nó yêu cầu các thông tin này, đảm bảo command luôn hợp lệ khi được tạo. Phía service, ta tạo lớp `PlaceOrderService` với phương thức `handle(PlaceOrderCommand $command): OrderId` . Bên trong, service sẽ sử dụng các repository cần thiết (đã được inject) để thực hiện nghiệp vụ: lấy thông tin sản phẩm từ `ProductRepository` , tạo đối tượng Order (entity domain) với dữ liệu từ command, tính toán tổng tiền, lưu Order qua `OrderRepository` , … rồi trả về ID đơn hàng hoặc một kết quả nào đó. Toàn bộ quá trình này không cần biết đến Request HTTP hay Session, chỉ làm việc với dữ liệu thuần từ command và các thành phần domain, infrastructure đã truyền vào. Điều này giúp use case tái sử dụng được ở các ngữ cảnh khác nhau - ví dụ cùng PlaceOrderService có thể dùng

cho trang web, cho API mobile, hoặc cho một script nội bộ – miễn là chuẩn bị được `PlaceOrderCommand` tương ứng.

Tích hợp vào Controller: Controller sẽ chịu trách nhiệm chuyển đổi từ chi tiết giao thức sang command. Trong phương thức controller (ví dụ `OrderController@placeOrder`), ta lấy dữ liệu từ HTTP request, khởi tạo `PlaceOrderCommand`:

```
$command = new PlaceOrderCommand(
$request->get('customer_id'),
$request->get('items'),
$request->get('email') // ... các tham số khác );
$orderId = $placeOrderService->handle($command);
```

Sau khi gọi service, controller nhận về OrderId (hoặc kết quả thành công/thất bại), rồi tạo Response (JSON hoặc chuyển trang). Toàn bộ logic nghiệp vụ (tạo đơn, tính tiền, trừ tồn kho, …) đã nằm gọn trong service. Cách làm này không chỉ làm controller ngắn gọn, dễ đọc, mà còn tách biệt hẳn tầng web với tầng ứng dụng. Nếu ngày mai ta muốn thêm kênh đặt hàng qua điện thoại (CLI command chẳng hạn), chỉ việc gọi `PlaceOrderService` với dữ liệu tương ứng, không cần nhân đôi logic.

Một lợi ích khác của command object là dễ validate và kiểm thử. Ta có thể kiểm tra tính hợp lệ của command trước khi đưa vào service (ví dụ dùng một lớp Validator cho PlaceOrderCommand). Trong unit test cho service, ta tạo giả một `PlaceOrderCommand` với dữ liệu mẫu và inject mock repository, sau đó gọi handle() xem kết quả, không cần dựng request giả hay môi trường web.

Kết hợp với Dependency Injection: Thông thường, PlaceOrderService sẽ cần sử dụng một số thành phần hạ tầng như repository, email sender, v.v. Thay vì sử dụng Service Locator (như gọi app()- >make('OrderRepository') trong Laravel), ta nên áp dụng Dependency Injection: truyền các dependency qua constructor của service. Ví dụ, PlaceOrderService có constructor yêu cầu OrderRepository , ProductRepository , Mailer … Khi khởi tạo service (thường tại Composition Root của ứng dụng), framework hoặc code khởi động sẽ inject những đối tượng cụ thể

(implementations) vào. Nhờ đó, service không phụ thuộc vào container toàn cục, và khi test có thể truyền vào các mock dễ dàng. (Chủ đề DI vs Service Locator sẽ được nói kỹ hơn ở chương sau).

Ví dụ minh họa `PlaceOrderService`:

```
final class PlaceOrderService
{
    private OrderRepository $orderRepo;
    private ProductRepository $productRepo;
    // ... có thể thêm EmailService, PaymentService nếu cần

    public function __construct(
        OrderRepository $orderRepo,
        ProductRepository $prodRepo /*,...*/
    ) {
        $this->orderRepo = $orderRepo;
        $this->productRepo = $prodRepo;
    }

    public function handle(PlaceOrderCommand $cmd): OrderId
    {
        // Lấy thông tin sản phẩm và kiểm tra tồn kho
        $product = $this->productRepo->getById($cmd->productId());

        if (!$product->isInStock($cmd->quantity())) {
            throw new OutOfStockException();
        }

        // Tính toán giá trị đơn hàng
        $orderAmount = $product->price()->multiply($cmd->quantity());

        // Lấy ID mới cho Order
        $orderId = $this->orderRepo->nextIdentity();
```

```
        // Tạo đối tượng Order domain
        $order = Order::create(
            $orderId,
            $cmd->customerId(),
            $product,
            $cmd->quantity(),
            $orderAmount
        );

        // Lưu order
        $this->orderRepo->save($order);

         // (Gửi email xác nhận nếu cần, có thể phát domain event hoặc gọi
EmailService)
        return $orderId;
    }
}
```

Đoạn code trên minh họa các bước chính: không hề truy cập Request hay Session, tất cả tương tác qua repository và domain object. Controller web chỉ lo lấy request thành `PlaceOrderCommand` và nhận orderId về để phản hồi.

Lợi ích: Mô hình **Application Service + Command** giúp use case trở nên "framework-agnostic" (không phụ thuộc framework). Chẳng hạn, ta có thể viết kịch bản test: tạo một PlaceOrderCommand với dữ liệu giả lập, gọi service và kiểm tra xem kết quả có đúng (order được lưu, email service gửi thư được gọi…). Không cần phải khởi động framework web hay database thực - ta có thể dùng fake repository in-memory và stub email service. Điều này thỏa mãn mục tiêu decoupling: logic core code chạy độc lập, không cần bối cảnh đặc biệt . Về lâu dài, code kiến trúc này cho phép ta dễ dàng thay đổi giao diện hoặc thêm giao tiếp mà không phải sửa logic nghiệp vụ (ví dụ chuyển từ MVC web sang REST API, hay thêm giao diện dòng lệnh) – tất cả đều dùng lại các application service bên dưới.

Tóm lại, việc tách use case ra thành Application Service và sử dụng Command object để truyền dữ liệu vào là một bước quan trọng hướng đến kiến trúc sạch. Nó tạo ra cầu nối rõ ràng giữa tầng giao tiếp (delivery layer) và tầng ứng dụng: data vào được gói trong command, logic xử lý nằm trong service, data ra (kết quả) trả về dạng đơn giản (thường là value object hoặc DTO). Các controller/handler trở nên nhẹ nhàng, chỉ còn nhiệm vụ chuyển đổi dữ liệu và gọi dịch vụ phù hợp . Chương sau, chúng ta sẽ đi sâu hơn vào cách quản lý dependency của các service này, so sánh Service Locator và Dependency Injection, cũng như cách tổ chức mã để việc khởi tạo đối tượng (composition root) diễn ra gọn gàng

Kiến trúc Phân lớp (Layered Architecture)

Giới thiệu

Kiến trúc phân lớp là một phong cách kiến trúc phần mềm cổ điển, trong đó ứng dụng được chia thành các tầng (layer) riêng biệt, mỗi tầng đảm nhiệm một nhóm chức năng có liên quan. Cách tiếp cận này xuất phát từ mô hình 3-tier (3 lớp) phổ biến:

- Presentation Layer (Lớp trình bày/Giao diện): xử lý hiển thị giao diện và tương tác người dùng (UI).
- Business Logic Layer (Lớp nghiệp vụ): chứa logic kinh doanh/cốt lõi của ứng dụng.
- Data Access Layer (Lớp truy cập dữ liệu): phụ trách làm việc với cơ sở dữ liệu hoặc hệ thống lưu trữ.

Mục tiêu chính của kiến trúc phân lớp là tách biệt các mối quan tâm (separation of concerns) - mỗi tầng giải quyết một khía cạnh riêng, giảm sự phụ thuộc chéo và trộn lẫn giữa chúng. Khi các tầng được thiết kế đúng cách và tuân thủ nguyên tắc, chúng tạo thành những “lớp đệm” bảo vệ phần bên dưới: tầng trên chỉ tương tác với tầng ngay dưới nó theo một giao diện nhất định, còn chi tiết thực hiện nằm sâu hơn sẽ được che giấu .

Kiến trúc phân lớp mang lại nhiều lợi ích:

- Giúp dễ dàng định vị code: bạn biết nên đặt lớp X vào tầng nào dựa trên trách nhiệm của nó . Ví dụ, mã xử lý giao diện sẽ ở tầng Presentation, tính toán nghiệp vụ ở tầng Business, truy vấn SQL ở tầng Data.

- Cho phép thay đổi công nghệ tại một tầng mà ít ảnh hưởng tới tầng khác (ví dụ đổi từ MySQL sang MongoDB ở Data Layer mà không cần sửa logic nghiệp vụ).
- Có thể xây dựng và kiểm thử từng tầng độc lập phần nào: ví dụ test logic nghiệp vụ mà không cần giao diện, hoặc thiết kế giao diện trước với dữ liệu giả lập.

Tuy nhiên, kiến trúc phân lớp truyền thống cũng tiềm ẩn rủi ro. Nếu không kiểm soát chặt chẽ, ranh giới giữa các tầng có thể bị xâm phạm - hay còn gọi là "rò rỉ logic" giữa các tầng. Ví dụ: code xử lý nghiệp vụ bị viết nhầm ở tầng giao diện (trong controller), hoặc ngược lại, tầng business lại đi thẳng xuống DB bỏ qua data layer. Không có cơ chế tự động nào ngăn chặn điều này; tất cả phụ thuộc vào kỷ luật của đội ngũ. Hơn nữa, thêm nhiều tầng cũng có mặt trái: codebase có nguy cơ biến thành "mã lasagna (một món ăn nổi tiếng của Ý, làm từ lớp bột nghìn lớp" - quá nhiều lớp chồng chéo khiến việc theo dấu luồng xử lý phức tạp . Do đó, việc chọn đúng số lượng tầng và xác định rõ vai trò cho từng tầng là rất quan trọng.

Trong chương này, chúng ta sẽ tìm hiểu một biến thể hiện đại của kiến trúc phân lớp, được áp dụng nhiều trong các ứng dụng web hướng Domain-Driven Design: phân chia thành các tầng **Domain, Application, Infrastructure** (và coi giao diện người dùng là một tầng riêng bên ngoài). Cách tiếp cận này nhấn mạnh đưa logic nghiệp vụ (domain) làm trọng tâm, tách rời khỏi hạ tầng kỹ thuật – phù hợp với những gì chúng ta đã xây dựng ở chương trước.

Vấn đề

Nhiều ứng dụng web truyền thống ban đầu không tuân thủ rõ ràng nguyên tắc phân lớp. Kết quả thường thấy là:

- **Code spaghetti:** Mọi lớp có thể gọi lẫn nhau không theo một quy ước nào. Ví dụ: controller trực tiếp truy cập database, model gọi ngược sang view... Code thiếu cấu trúc, rất khó nắm luồng tổng thể. Tình trạng này đặc biệt hay gặp ở các legacy project không có định hướng kiến trúc ngay từ đầu .
- **Phát triển "từ dưới lên":** Một hệ quả của việc không chú trọng domain là nhiều dự án bắt đầu bằng thiết kế database, tạo các model từ bảng, rồi viết logic nghiệp vụ phụ thuộc chặt vào cấu trúc DB đó. Cách làm này (theo mô hình Active Record chẳng hạn) khiến domain model trở thành "nô lệ" cho database, thay vì phản ánh đúng nghiệp vụ. Service (nếu có) cũng bị phụ thuộc vào tầng truy cập dữ liệu . Sự phụ thuộc ngược (nghiệp vụ

phụ thuộc data layer) khiến bất kỳ thay đổi ở DB hay công nghệ lưu trữ cũng có thể tác động ngược lên logic nghiệp vụ.

- **Khó thay đổi và mở rộng:** Khi logic nghiệp vụ trộn lẫn với code hạ tầng, muốn chỉnh sửa quy tắc kinh doanh (business rule) đòi hỏi đụng chạm cả những đoạn code giao tiếp DB, framework. Nguy cơ cao gây bug ngoài ý muốn. Đồng thời, thêm tính năng mới dễ dẫn đến sao chép mã hoặc sửa cùng lúc ở nhiều nơi.
- **Kiểm thử phức tạp:** Tương tự như đã đề cập, nếu không có ranh giới tầng rõ ràng, việc viết test sẽ đối mặt với việc thiết lập nhiều thứ (ví dụ: muốn test một hàm tính toán đơn giản nhưng hàm nằm trong lớp controller, vốn cần HTTP request giả lập, hoặc tệ hơn gọi cả DB). Điều này làm test chậm và dễ flaky (chạy lúc đúng lúc sai).

Tựu trung, thiếu kiến trúc phân lớp biến ứng dụng thành một khối đặc khó tách rời. Ngược lại, phân lớp đúng đắn sẽ đem lại "trật tự", giúp mã nguồn dễ bảo trì hơn nhiều. Câu hỏi là: chia thành những lớp nào để tối ưu cho ứng dụng web theo hướng domain?

Giải pháp

Giải pháp được đề xuất dựa trên kinh nghiệm Domain-Driven Design và Clean Architecture là phân chia ứng dụng web thành 3 tầng chính:

- Domain Layer (Tầng miền nghiệp vụ)
- Application Layer (Tầng ứng dụng)
- Infrastructure Layer (Tầng hạ tầng)

Ngoài ra, ta có thể xem Presentation Layer (Tầng giao diện) như một lớp nằm trên cùng, tương tác với người dùng, nhưng thường tầng này thuộc phần Infrastructure (vì dùng công nghệ framework web). Mô hình này gần tương đồng với Clean Architecture hoặc Hexagonal Architecture, nhấn mạnh luồng phụ thuộc một chiều: từ ngoài (UI, Infrastructure) hướng vào trong (Domain). Tức là, code tầng ngoài có thể gọi code tầng trong, nhưng ngược lại thì không - tầng Domain hoàn toàn độc lập, không biết gì về những thứ bên ngoài nó. Chúng ta sẽ lần lượt xem xét vai trò của từng tầng:

Tầng Domain (Miền nghiệp vụ)

Domain Layer là trái tim của ứng dụng:nơi chứa mọi logic nghiệp vụ cốt lõi, mô hình hóa các khái niệm, quy tắc, hành vi của bài toán thực tế mà phần mềm giải quyết. Tầng này tương ứng

với những gì chúng ta đã thiết kế ở chương trước: các **Entity, Value Object, Domain Service, Domain Event,**... tất cả đều tập trung thể hiện nghiệp vụ và không phụ thuộc hạ tầng. Thành phần chính trong tầng Domain:

- **Entities & Aggregates:** Các đối tượng có định danh, trạng thái và vòng đời riêng, đại diện cho các thực thể kinh doanh. Mỗi Entity/Aggregate đảm bảo các bất biến nghiệp vụ và cung cấp hành vi liên quan.
- **Value objects:** Các đối tượng giá trị bất biến, mô tả những giá trị đặc thù (như đơn vị tiền tệ, phần trăm thuế, địa chỉ).
- **Domain Services:** Nếu có nghiệp vụ nào không tự nhiên thuộc về một entity cụ thể, ta có thể tạo domain service – một lớp thuần túy chứa logic đó, thường là các hàm tĩnh hoặc hàm thể hiện không trạng thái.
- **Domain Events:** Các sự kiện phát sinh trong domain khi trạng thái thay đổi (như đã nói: `OrderPlaced, ProductOutOfStock`,...). Chúng cũng nằm ở domain dưới dạng các class sự kiện.
- **Repository interfaces:** Định nghĩa các cổng (interface) để tương tác lưu trữ cho mỗi Aggregate. Chúng nằm ở domain nhưng chỉ là giao diện, không chứa logic truy cập cơ sở dữ liệu cụ thể .
- **Factory:** (tuỳ chọn) Các lớp hoặc phương thức dùng để tạo các đối tượng phức tạp (nhất là Aggregate) một cách thống nhất.

Điểm đặc trưng của domain layer:

- **Không phụ thuộc framework/hạ tầng:** Domain hoàn toàn "thuần" PHP (hoặc ngôn ngữ bạn dùng), không import hay gọi đến code của Laravel, không truy vấn database trực tiếp, không gửi HTTP, không log file hệ thống. Domain model "sạch" và chỉ chứa logic của nó . Nhờ vậy, domain có thể tồn tại độc lập, thậm chí bạn có thể tách nó thành một thư viện riêng mà các ứng dụng khác (dù khác framework) có thể tái sử dụng.
- **Kiểm thử dễ dàng:** Vì không dính bên ngoài, bạn có thể viết unit test chạy toàn bộ domain logic trong memory, rất nhanh và ổn định. Các test domain không cần boot Laravel hay bất kỳ service nào khác, đảm bảo tính dễ đoán trước và tối ưu tốc độ.
- **Được gọi bởi tầng Application:** Domain tự thân không khởi chạy. Nó như một "lõi" bị động, chỉ khi có lớp trên gọi tới (qua service hoặc qua repository) thì code domain mới thực thi. Điều này tuân theo nguyên tắc Hollywood Principle: "Don't call us, we'll call you" - domain không gọi tầng khác, tầng khác gọi nó. Tóm lại, Domain layer chứa đựng toàn bộ tri thức nghiệp vụ và là nền móng cho hệ thống.

Tầng Application (Ứng dụng)

Application Layer nằm ngay bên ngoài Domain. Tầng này chịu trách nhiệm điều phối các hoạt động nghiệp vụ để thực hiện yêu cầu người dùng hoặc yêu cầu từ hệ thống ngoài. Hiểu đơn giản, application layer sử dụng các năng lực của domain để hoàn thành một trường hợp sử dụng (use case) cụ thể. Những thành phần chính trong application layer:

- **Application Services / Use Case Handlers:** Mỗi chức năng (use case) quan trọng của hệ thống được triển khai thành một dịch vụ ứng dụng (như class `PlaceOrderHandler` hoặc `OrderService` với phương thức `placeOrder`). Dịch vụ này nhận lệnh hoặc yêu cầu (thường ở dạng DTO, ví dụ PlaceOrderCommand) và tiến hành tương tác với domain: gọi repository để lấy Entity, gọi phương thức entity để thay đổi trạng thái, tạo entity mới, v.v., rồi lưu lại qua repository, phát sự kiện domain nếu có.
- **Command/Query Objects:** Nhiều kiến trúc đưa ra mô hình CQS/ CQRS, trong đó Command đại diện cho yêu cầu thay đổi trạng thái (có side effect), Query đại diện cho yêu cầu đọc dữ liệu. Application layer có thể bao gồm các Command object (DTO chứa dữ liệu để thực hiện hành động) và Command Handler tương ứng; tương tự cho Query (một số nơi dùng Query object + Query Handler, hoặc gộp vào Service).
- **Facade/Orchestrator:** Đôi khi, application service đóng vai trò như một facade tập hợp nhiều bước hoặc gọi nhiều service con. Ví dụ trong một giao dịch phức tạp, application service có thể gọi nhiều phương thức domain khác nhau để hoàn tất một tác vụ lớn. Nó điều phối luồng đi, đảm bảo mọi thứ xảy ra đúng thứ tự.

Đặc điểm của application layer:

- **Phụ thuộc domain, không ngược lại:** Application layer biết về domain (gọi các interface repository, entity, domain service) nhưng domain hoàn toàn không biết về application layer. Điều này đúng theo Dependency Rule: “Code ở tầng trong không được phụ thuộc vào code ở tầng ngoài” .
- **Không chứa logic nghiệp vụ chi tiết:** Application service có thể có một chút logic điều kiện, nhưng phần lớn logic kinh doanh đã được bao gói ở domain. Nếu bạn thấy application service bắt đầu có nhiều tính toán phức tạp, có thể xem xét đẩy xuống domain (vd thêm phương thức cho Entity hoặc domain service).
- **Xử lý transaction và tích hợp:** Application layer thường đảm nhận việc bắt đầu và kết thúc Transaction (ví dụ mở transaction database, commit sau khi hoàn thành tất cả bước). Ngoài ra, nó cũng là nơi triển khai việc gọi các service ở tầng hạ tầng nếu cần

(mặc dù lý tưởng thì mọi thứ hạ tầng đều qua interface, nhưng application có thể gọi thẳng một số thứ như gửi email qua adapter nếu nhỏ).

- **Chuẩn bị dữ liệu cho Presentation:** Đôi khi, application service sau khi thực hiện xong use case còn trả về một DTO kết quả hoặc khởi tạo một View model để tầng trình bày có dữ liệu sẵn sàng. Ví dụ: một service `GetOrderDetailQuery` có thể trả về một DTO `OrderDetailData` để controller dùng render ra view.
- **Phân chia module theo use case:** Ở cấp độ kiến trúc, ta có thể thấy application layer như tập hợp của nhiều "bộ xử lý use case". Mỗi use case thuộc một Bounded Context hay phân hệ có thể nhóm thành package riêng bên trong tầng application.Tóm lại, application layer như một "lớp keo" kết nối nhu cầu bên ngoài với năng lực bên trong (domain). Nó không làm thay domain, mà gọi domain để làm việc.

Tầng Infrastructure (Hạ tầng)

Infrastructure Layer chứa tất cả các thành phần kết nối ứng dụng với thế giới bên ngoài, cung cấp kỹ thuật triển khai cụ thể cho các interface trừu tượng ở tầng trên. Đây là lớp ngoài cùng, cũng thường là phần code đồ sộ nhất vì bao gồm nhiều thứ linh tinh: từ framework, cơ sở dữ liệu, cho tới tích hợp hệ thống khác. Thành phần trong tầng Infrastructure có thể kể như:

- **Framework & Drivers:** code liên quan đến framework web (Laravel) như lớp Controller, Form Request, Routes, hoặc Console Command, Jobs queue. Những thứ này làm nhiệm vụ nhận input từ HTTP, CLI,... và chuyển vào application layer.
- **ORM & Database:** Triển khai các repository bằng công nghệ cụ thể. Ví dụ: lớp `EloquentOrderRepository` dùng Eloquent, hoặc một lớp Repository khác dùng PDO/Query Builder thuần. Cũng có thể bao gồm các Migration, seeder - nói chung mọi thứ để giao tiếp và quản lý DB.
- **External Service Adapters:** Code để gọi các dịch vụ ngoài qua API, hoặc tích hợp message broker, tệp tin. Ví dụ: tích hợp gửi email (SMTP, Mailgun... trong lớp MailService), gọi API thanh toán (class `StripePaymentGateway`).
- **Infrastructure Services:** Những dịch vụ kỹ thuật như logging, authentication, caching. Trong Laravel, nhiều cái trong số này nằm sẵn (Auth, Cache facades), ta coi chúng thuộc hạ tầng.
- Implementations của Interfaces: Tất cả interface ở domain hoặc application cần được hiện thực ở đây. Như OrderRepository -> EloquentOrderRepository , hay interface

NotificationService -> MailNotificationService (gửi email qua Laravel Mailable). Đặc điểm tầng Infrastructure:

- **Phụ thuộc vào application/domain:** Lớp hạ tầng sẽ triển khai các interface do tầng trong định nghĩa, nên nó phụ thuộc ngược lại vào các định nghĩa đó (nhưng đây là phụ thuộc cho compile-time, runtime thì injection giải quyết). Theo nguyên tắc DIP, ta hoàn toàn chấp nhận việc infrastructure phụ thuộc domain (vì domain chỉ là interface, không kéo phụ thuộc cụ thể nào) .
- **Chứa logic kỹ thuật, không chứa logic nghiệp vụ:** Code trong infra dù có phức tạp (ví dụ truy vấn SQL tối ưu) nhưng không quyết định các rule nghiệp vụ. Nếu có logic if/else ở đây thì thường liên quan tới kỹ thuật (vd retry kết nối, format lại dữ liệu bên ngoài), chứ không phải rule nghiệp vụ.
- Kiểm thử dạng integration: Vì hầu hết code hạ tầng đòi hỏi có môi trường bên ngoài (DB, service) nên unit test thuần cho chúng ít ý nghĩa. Thay vào đó, ta viết integration test - ví dụ test `EloquentOrderRepository` bằng cách kết nối real database, xem lưu và đọc có khớp không . Các test này chạy chậm hơn (vì thực sự truy cập I/O), nhưng cần thiết để an tâm rằng phần kết nối ngoài không lỗi. Như vậy, Infrastructure layer là nơi ta "ít clean nhất" với các chi tiết nền tảng, nhưng nhờ có tầng này tách riêng, hai tầng bên trong có thể "clean - sạch sẽ" tập trung vào nghiệp vụ.

Quan hệ giữa các tầng:

Trong mô hình này, ta tuân thủ nguyên tắc chỉ phụ thuộc vào tầng bên trong. Cụ thể:

- Domain độc lập hoàn toàn.
- Application có thể phụ thuộc Domain (gọi phương thức entity, dùng interface repo).
- Infrastructure có thể phụ thuộc Application (ví dụ một Listener sự kiện domain được đăng ký trong tầng infra có thể gọi Application service khác) và phụ thuộc Domain (triển khai interface, sử dụng các Value Object định nghĩa ở domain).
- Tầng giao diện (Presentation), nếu tách riêng, sẽ phụ thuộc Application (gọi service) hoặc trực tiếp Domain (trong trường hợp hiển thị read model). Sơ đồ đơn giản của kiến trúc phân lớp Domain-Application-Infrastructure như sau (mũi tên biểu thị hướng phụ thuộc compile-time từ ngoài vào trong):

[Giao diện người dùng] -> [Application] -> [Domain] <- [Infrastructure] (Controller) (Service) (Entity, logic) (ORM, Framework)

Nhìn vào luồng trên: Yêu cầu từ bên ngoài -> qua lớp giao diện (Controller) -> gọi vào Application Service -> Application thao tác Domain -> Domain có thể phát sự kiện ->

Application/Infrastructure lắng nghe để tiếp tục tương tác (nếu cần) -> cuối cùng kết quả quay ra Presentation để hiển thị. Ở chiều ngược lại, Domain có thể yêu cầu một hoạt động hạ tầng (như lưu DB, gửi email), nhưng thay vì tự gọi, nó thông qua interface (port) do Domain định nghĩa, Application cầm interface đó gọi xuống Infrastructure (adapter) thực hiện rồi trả kết quả lên.

Ví dụ minh hoạ Laravel/PHP

Giả sử trong thư mục app/ của Laravel, ta sắp xếp:

```
app/
├── Domain/
│   └── Order/
│       ├── Model/
│       │   ├── Order.php
│       │   ├── OrderLine.php
│       │   ├── Product.php
│       │   └── Price.php
│       ├── Event/
│       │   ├── OrderPlaced.php
│       │   └── OrderCancelled.php
│       ├── Repository/
│       │   └── OrderRepository.php
│       └── Service/
│           └── DomainServices/        # (nếu cần)
│
├── Application/
│   └── Order/
│       ├── Command/
│       │   └── PlaceOrderCommand.php
│       ├── Handler/
│       │   └── PlaceOrderHandler.php
│       └── Listener/
│           └── SendConfirmationOnOrderPlaced.php
```

```
│
├── Infrastructure/
│   ├── Persistence/
│   │   ├── EloquentOrderModel.php
│   │   └── EloquentOrderRepository.php
│   ├── Notifications/
│   │   └── MailNotificationService.php
│   ├── Http/
│   │   └── Controllers/
│   │       └── OrderController.php
│   └── Providers/
│       └── OrderServiceProvider.php
```

Cấu trúc trên chỉ là gợi ý để hình dung:

- Thư mục Domain/Order chứa mọi thứ liên quan domain của đặt hàng: model (entity, value), sự kiện, interface repository, thậm chí dịch vụ domain (nếu logic phức tạp).
- Thư mục Application/Order chứa các lớp điều phối: lệnh (Command) và bộ xử lý (Handler), và cả các listener ứng dụng. Lưu ý, listener ở đây xử lý sự kiện domain nhưng có thể vẫn dùng tài nguyên hạ tầng (gửi mail). Chúng ta tạm xếp vào Application cho thấy logic xử lý thuộc nghiệp vụ ứng dụng (có thể đặt trong Infrastructure nếu coi gửi mail thuần kỹ thuật).
- Thư mục Infrastructure/Persistence cài đặt repository với Eloquent. Có thể có model Eloquent riêng hoặc sử dụng luôn Model chung của Laravel, tùy lựa chọn. Ở đây minh họa tạo OrderModel (kế thừa Eloquent) để ánh xạ bảng orders , order_items ... Repository sẽ dùng nó.
- Infrastructure/Notifications triển khai một service gửi thông báo qua email (giả sử domain định nghĩa interface NotificationService, hoặc ta dùng trực tiếp luôn adapter này trong listener).
- Infrastructure/Http có controller Web. Tầng này nhận request và chuyển cho Application. (Cũng có thể có Infrastructure/Console cho command-line, v.v.)
- Infrastructure/Providers có ServiceProvider của Laravel để cấu hình DI.

Điểm đáng chú ý:

Tại OrderServiceProvider , ta sẽ đăng ký các implement cho interface:

```
$this->app->bind( \App\Domain\Order\Repository\OrderRepository::class,
\App\Infrastructure\Persistence\EloquentOrderRepository::class );
```

Nhờ đó, ở runtime, khi `PlaceOrderHandler` cần OrderRepository , Laravel container sẽ inject instance EloquentOrderRepository . Đây chính là cách chúng ta "nối" tầng Infrastructure với Application/Domain một cách lỏng lẻo, tuân theo DIP. Tiếp theo, luồng hoạt động:

- HTTP Request tới /orders/place -> Router chỉ định OrderController@placeOrder . - Controller (Infrastructure) nhận request, tạo PlaceOrderCommand từ dữ liệu đầu vào rồi gọi $this->handler->handle($command) . PlaceOrderHandler nằm ở Application, đã được container inject (nhờ khai báo trong controller constructor).
- PlaceOrderHandler thực thi: dùng OrderRepository (interface) để lưu/đọc Order, gọi domain logic (như $order- >confirm()), phát sự kiện domain. - OrderRepository thực tế là EloquentOrderRepository (Infrastructure) nên khi save sẽ dùng Eloquent model ghi DB.
- Domain Event OrderPlaced được phát đi. Laravel EventServiceProvider đã đăng ký `SendConfirmationOnOrderPlaced` lắng nghe nó.
- Listener SendConfirmationOnOrderPlaced (Application layer) chạy, sử dụng MailNotificationService (Infrastructure, có thể inject qua constructor) để gửi email.
- Cuối cùng, controller nhận kết quả (id đơn hàng) từ handler và trả về view/JSON cho người dùng.

Quy trình trên cho thấy rõ ràng: Domain (Order, OrderRepository interface, ...): không hề biết request hay DB. Application (Handler, Listener): điều phối logic, cũng không biết chi tiết DB hay email, chỉ biết gọi qua interface hoặc dịch vụ đã đăng ký. - Infrastructure: đảm nhiệm những công việc cụ thể như thật sự lưu DB, thật sự gửi email, hiển thị web.

So sánh với cách truyền thống (không phân lớp):

Không phân lớp: bạn có thể viết toàn bộ logic tạo đơn hàng ngay trong Controller - từ validate input, tạo Order (Eloquent model), lưu, gửi mail, trả view. Code ngắn hạn có thể nhanh, nhưng dài hạn rất khó bảo trì. Unit test hầu như không thể vì controller phụ thuộc quá nhiều thứ. Phân lớp: code dài hơn, chia ra nhiều file, nhưng mỗi file có một trách nhiệm rõ. Controller chỉ ủy thác công việc. Service lo nghiệp vụ chính. Mỗi thành phần có thể thay đổi độc lập: muốn đổi cách lưu DB

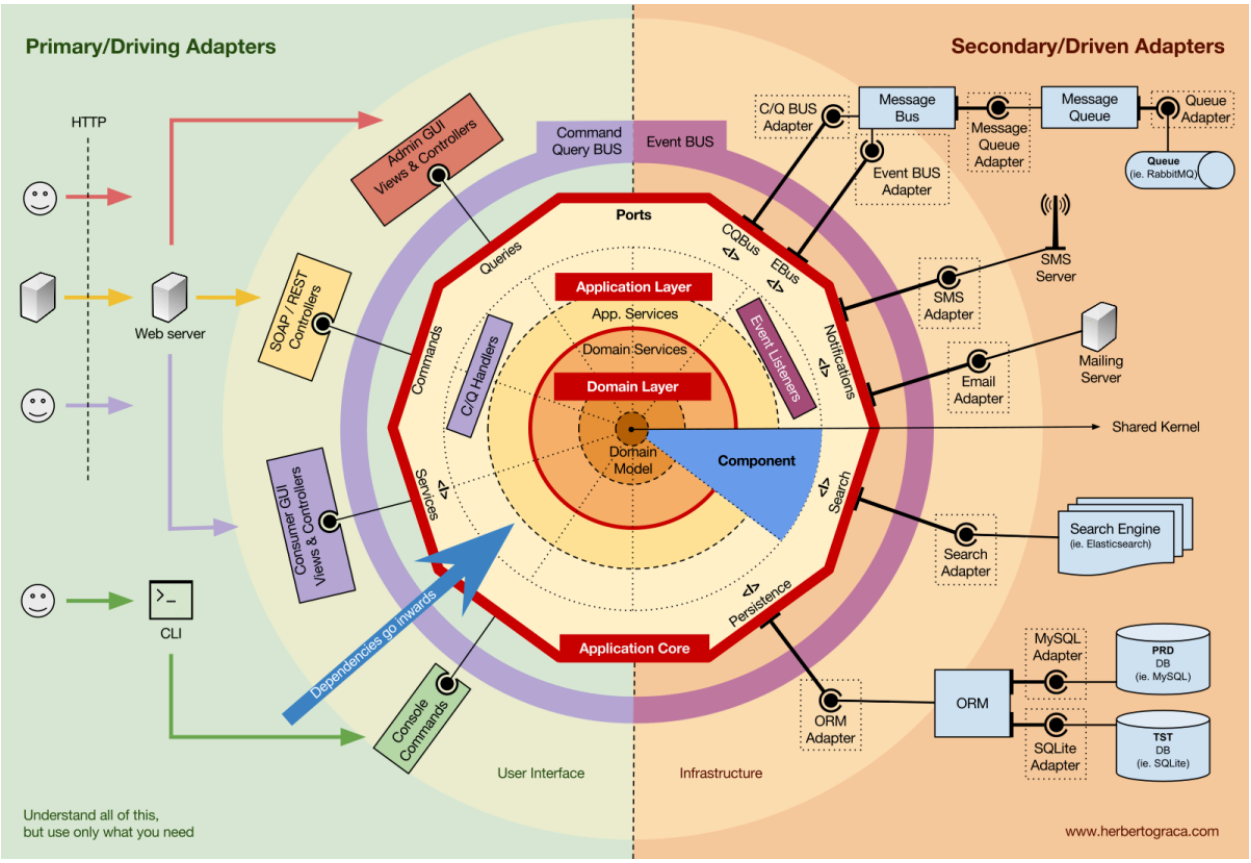
chỉ cần viết lớp repository khác và bind, không đụng vào service hay controller; muốn thêm thông báo SMS khi `OrderPlaced` chỉ việc viết thêm listener mới.
Lưu ý thực tế: Không phải lúc nào cũng cần tạo quá nhiều lớp cho mọi thứ. Đôi khi với các tác vụ rất đơn giản, ta có thể gộp bớt cho đỡ rối. Tuy nhiên, với những module phức tạp và lõi hệ thống, kiến trúc phân lớp này giúp mã của bạn "trưởng thành" hơn, dễ dàng thích ứng với yêu cầu mới.

Ports and Adapters (Hexagon Architecture - Kiến trúc Hình lục giác)

Giới thiệu

Trong chương trước, chúng ta đã thấy kiến trúc phân lớp giúp tách domain logic khỏi hạ tầng. Kiến trúc hình lục giác (Hexagonal Architecture) - còn được gọi là kiến trúc Ports and Adapters - đưa ý tưởng này đi xa hơn, tạo ra một cách tiếp cận linh hoạt để kết nối ứng dụng với các hệ thống bên ngoài. Kiến trúc này được đề xuất bởi Alistair Cockburn vào năm 2005, với mục tiêu làm cho ứng dụng không phụ thuộc vào framework, cơ sở dữ liệu hay bất kỳ công nghệ cụ thể nào. Hexagonal Architecture được xem là một bước tiến hoá từ kiến trúc phân lớp, giúp chuyển trọng tâm vào domain (logic nghiệp vụ) - thứ quan trọng nhất - thay vì để tâm quá nhiều đến framework hay database . Thay vì xem ứng dụng là tập hợp các tầng cứng nhắc, kiến trúc hình lục giác ví ứng dụng như một lục giác với phần lõi ở giữa và các cổng (ports) trên các mặt, cho phép kết nối với thế giới bên ngoài qua những bộ chuyển đổi (adapter).

Thuật ngữ "hình lục giác" chỉ mang tính ẩn dụ: một lục giác có 6 cạnh, tượng trưng cho việc ứng dụng của bạn có thể có nhiều "cổng" để giao tiếp (không nhất thiết đúng 6 cổng, số lượng tuỳ nhu cầu). Điểm mấu chốt là mọi tương tác từ bên ngoài đều phải thông qua một cổng (port) và một bộ chuyển đổi (adapter) để đến được lõi ứng dụng, và ngược lại, mọi yêu cầu từ lõi đi ra bên ngoài cũng qua cổng & adapter tương ứng . Hãy hình dung: nếu coi ứng dụng như một thiết bị, thì port giống như cổng kết nối (như cổng USB trên máy tính), còn adapter là phần "đầu nối" cụ thể cắm vào cổng đó (như dây sạc USB, USB drive...). Máy tính không quan tâm bạn cắm thiết bị gì, miễn nó tuân thủ giao thức cổng USB. Tương tự, ứng dụng không quan tâm chi tiết bên ngoài, miễn adapter tuân thủ interface của port.



Vấn đề

Kiến trúc phân lớp đã giúp giảm phụ thuộc, nhưng vẫn còn vài hạn chế khi đối mặt với các thay đổi và mở rộng phức tạp:

- **Phụ thuộc ngầm giữa core và hạ tầng:** Dù chúng ta có tách domain và infra, vẫn có trường hợp lập trình viên vô tình dùng trực tiếp một lớp hạ tầng trong core vì tiện lợi (vi phạm quy tắc). Không có cơ chế kỹ thuật nào ép buộc điều này ngoài kỷ luật. Điều này có thể làm domain "rò rỉ" phụ thuộc.
- **Thêm kênh tương tác mới khó khăn:** Giả sử ban đầu ứng dụng chỉ có giao diện web, sau đó cần thêm giao diện dòng lệnh (CLI) hoặc tích hợp một đối tác qua API. Nếu không thiết kế sẵn cổng, nhiều khả năng bạn phải sửa nhiều chỗ trong application để hỗ trợ kênh mới (ví dụ phải thêm logic trong service để phân biệt request từ web hay CLI). Cấu trúc phân lớp cơ bản chưa chỉ rõ cách xử lý nhiều nguồn tương tác đồng thời.

- **Khó thử nghiệm toàn diện:** Mặc dù domain đã test đơn vị tốt, nhưng test tầng application có thể vẫn chậm nếu nó gọi thẳng hạ tầng thực (vd test service vẫn gọi DB qua repository thật). Cần một cách để thay thế linh hoạt các thành phần bên ngoài bằng giả lập trong bối cảnh test mà core không cần biết.
- **Tích hợp hệ thống phức tạp:** Trong kiến trúc lớn, một ứng dụng có thể gọi ứng dụng khác (liên bounded context). Nếu không có quy ước rõ, code tích hợp dễ làm lộn xộn domain (ví dụ: service A gọi thẳng HTTP client tới service B ngay trong logic, làm domain A phụ thuộc giao thức cụ thể). Những vấn đề này đòi hỏi một giải pháp kiến trúc cho phép cô lập hoàn toàn phần lõi ứng dụng, đồng thời linh hoạt cắm thêm các "đầu nối" mới khi cần mà không động chạm code lõi.

Giải pháp

Hexagonal Architecture giải quyết bằng cách giới thiệu hai khái niệm cốt lõi: Port (Cổng) và Adapter (Bộ chuyển đổi).

Port - Cổng giao tiếp trừu tượng

Port đại diện cho một điểm tương tác giữa ứng dụng với thế giới bên ngoài, được định nghĩa một cách trừu tượng. Port có thể là:

- **Port đầu vào (Primary/Input Port):** Cách mà ứng dụng nhận yêu cầu thực thi use case. Ví dụ: Giao diện người dùng (web UI, mobile app), API bên ngoài gọi vào, một script dòng lệnh, hoặc thậm chí là bộ kiểm thử tự động. Mỗi kênh vào nên có một port tương ứng .
- **Port đầu ra (Secondary/Output Port):** Cách ứng dụng gửi dữ liệu/ yêu cầu ra ngoài. Ví dụ: Lưu trữ dữ liệu (Persistence port), gửi thông báo ra ngoài (Notification port), gọi dịch vụ bên ngoài (External Service port) .

Port thường được hiện thực trong code dưới dạng interface hoặc một hợp đồng nào đó (có thể chỉ là một protocol quy ước). Bản thân port không làm gì, nó chỉ định nghĩa "muốn tương tác thì phải theo chuẩn này". Trong domain của chúng ta, các `Repository interface` chính là ví dụ của Output Port (ứng dụng cần lưu/đọc dữ liệu thì gọi qua repository interface, còn ai thực hiện không quan tâm) . Tương tự, một interface `NotificationService` có thể coi là một Output Port cho chức năng gửi thông báo.

Đối với input port, có thể không cần interface cụ thể mà hiểu là mỗi cách vào có một thành phần xử lý riêng (như Controller web đóng vai trò cổng nhận HTTP, một lớp `ConsoleCommand` đóng vai trò cổng nhận lệnh CLI). Tuy nhiên, ta có thể trừu tượng hoá cao hơn: ví dụ định nghĩa một interface OrderPlacementUI có phương thức askForOrderDetails() rồi implement nó cho WebUI, CLIUI... Điều này phụ thuộc vào thiết kế, nhưng tinh thần chung: ứng dụng không phụ thuộc trực tiếp vào chi tiết nguồn vào, mà thông qua một hợp đồng.

Một port có thể rộng hoặc hẹp tuỳ nhu cầu thiết kế. Cockburn có nói, nếu cực đoan, mỗi use case là một port riêng . Thực tế hay làm gọn lại: ta tạo port theo nhóm chức năng (vd một PersistencePort chung cho mọi entity, hoặc tách theo loại tác nhân: UserInterfacePort , PartnerApiPort ,...). Điều quan trọng là xác định đúng những cổng cần thiết để ứng dụng giao tiếp, từ đó chuẩn hoá sự tương tác.

Adapter - Bộ chuyển đổi cụ thể

Adapter là phần code cụ thể kết nối port với công nghệ hoặc hệ thống thực tế. Mỗi port sẽ có ít nhất một adapter để nó hoạt động được. Ví dụ: - Port "`UserInterface`": có adapter là WebController (xử lý HTTP) hoặc ConsoleCommand (xử lý CLI). - Port "Persistence": có adapter là `EloquentOrderRepository` (dùng MySQL) hoặc một `MongoOrderRepository` . - Port "Notifications": có adapter EmailNotification (gửi email qua SMTP) hoặc SlackNotification (gửi tin nhắn qua Slack API). - Port "External Payment": adapter StripePaymentClient (gọi API của Stripe).

Adapter chứa code rất cụ thể: xử lý giao thức, chuyển đổi dữ liệu qua lại giữa format bên ngoài và format bên trong ứng dụng. Do đó adapter thuộc về tầng hạ tầng (Infrastructure) . Adapter phụ thuộc vào port tương ứng: tức là adapter implement interface của port (đối với output port), hoặc adapter gọi phương thức của port (đối với input port). Nhờ tuân thủ DIP, adapter không đòi hỏi thay đổi gì ở core – bạn có thể viết thêm adapter mới bất kỳ lúc nào miễn là port đã được định nghĩa.

Quan hệ giữa port và adapter theo nguyên tắc 1-n: Mỗi port có thể có nhiều adapter khác nhau cùng cắm vào. Thường sẽ có: - Ít nhất một adapter thực để chạy ứng dụng thực tế (ví dụ database thật, UI thật). - Ít nhất một adapter giả để phục vụ kiểm thử (in-memory database, giả

lập gửi mail...) . Đây chính là điểm mạnh ban đầu của hexagonal: viết test nhanh bằng cách dùng adapter in-memory thay cho adapter thật chậm chạp. Thực tế, Cockburn đề xuất kiến trúc này chủ yếu để hỗ trợ testing hiệu quả ngay từ đầu .

Một lần nữa, lấy hình ảnh cổng USB: bạn có thể cắm nhiều loại thiết bị (adapter) khác nhau vào cùng cổng USB: chuột, bàn phím, USB drive, máy in,... Tương tự, ứng dụng có thể có nhiều adapter cho một port. Thậm chí cùng chức năng có adapter khác nhau song song (vd hai adapter lưu trữ: MySQL và MongoDB chạy đồng thời để thử nghiệm). Ứng dụng không cần biết, nó chỉ tương tác qua cổng chung.

Vận dụng Ports & Adapters vào kiến trúc phân lớp

Hexagonal Architecture không thay thế Layered Architecture mà bổ sung cho nó. Thực ra, ta có thể xem phần Domain + Application chính là lõi của lục giác, còn Infrastructure chính là các adapter bao quanh. Sự khác biệt là hexagonal đề xuất ta nghĩ rõ hơn về các loại interaction. Ở kiến trúc phân lớp, ta nói "Infrastructure layer làm tất cả việc kết nối ngoài", còn hexagonal chi tiết hoá: liệt kê rõ những "kết nối ngoài" đó là gì (UI, DB, Mail, Payment, Logging, etc.) và tạo cơ chế cắm rút cho từng cái.

Do đó, để áp dụng hexagonal, chúng ta làm các bước:

- **Xác định các port cần có của ứng dụng:** Dựa vào use case và phụ thuộc bên ngoài: Ứng dụng cung cấp những cách nào để người/ hệ thống khác tương tác? (UI, API, CLI => các input ports). - Ứng dụng cần sử dụng những tài nguyên/ dịch vụ bên ngoài nào? (DB, cache, email, hệ thống khác => output ports).
- **Định nghĩa interface hoặc hợp đồng cho mỗi port đó trong core (domain/application)**. Ví dụ: OrderRepository (port lưu trữ Order), CustomerRepository , NotificationService , PaymentGateway , etc. Với input port, có thể là danh sách các phương thức service application (hoặc interface facade) mà adapter ngoài sẽ gọi.
- **Triển khai adapter tương ứng trong tầng Infrastructure:** Implement các repository interface bằng công nghệ thực (MySQL, Mongo). Viết controller/ CLI command gọi vào application service (đóng vai trò adapter cho input). Viết lớp tích hợp dịch vụ ngoài (triển khai interface PaymentGateway bằng Guzzle HTTP client chẳng hạn). Viết adapter giả (FakeRepository, FakePaymentGateway) dùng cho test hoặc dev offline.
- Kết nối bằng Dependency Injection: Sử dụng container để bind interface port -> class adapter cụ thể. Với input adapter (như controller), đảm bảo nó gọi đúng service ở core.

Ví dụ minh hoạ Laravel/PHP

Quay lại module Đơn hàng (Order) của chúng ta, hãy xác định các port:

- **User Interface Port:** cho phép người dùng tạo đơn hàng. Ở ứng dụng ta, port này sẽ được thực hiện bởi adapter là OrderController (Web UI). Nếu sau này có CLI (ví dụ lệnh import đơn hàng từ file), đó sẽ là adapter thứ hai cho port này.
- **Persistence Port:** để lưu và lấy Order từ cơ sở dữ liệu. Port này chính là interface OrderRepository . Adapter hiện tại: `EloquentOrderRepository` (dùng MySQL). Adapter khác cho test: `InMemoryOrderRepository` .
- **Notification Port:** để gửi thông báo cho khách sau khi đặt hàng. Ta định nghĩa một interface `OrderNotification` với phương thức `sendOrderConfirmation(Order $order)` . Adapter thật: `EmailOrderNotification` (gửi email qua hệ thống thư), adapter giả: `LogOrderNotification` (ghi log hoặc bỏ qua, dùng khi test).
- Có thể còn port khác như Payment Port nếu khi đặt hàng cần gọi cổng thanh toán, nhưng trong ví dụ này đơn giản bỏ qua. Cấu trúc thư mục theo kiểu hexagonal có thể như sau:

```
app/
├── Domain/
│   └── Order/
│       ├── Order.php                       # (Entity)
│       ├── ValueObjects/                   # (Các đối tượng giá trị)
│       ├── Event/                          # (Domain Events)
│       └── Repository/
│           └── OrderRepository.php         # (Port: Persistence)
│
├── Application/
│   └── Order/
│       ├── PlaceOrderHandler.php           # (Use case logic)
│       └── Port/
│           └── OrderNotification.php       # (Port: Notification)
│
└── Infrastructure/
    ├── Order/
```

```
    │   ├── UI/
    │   │   ├── Http/
    │   │   │   └── OrderController.php          # (Adapter for UI port - Web)
    │   │   └── Console/
    │   │       └── ImportOrderCommand.php       # (Adapter for UI port - CLI, giả sử có)
    │   │
    │   ├── Persistence/
    │   │   ├── EloquentOrderRepository.php          # (Adapter for Persistence port - MySQL)
    │   │   └── InMemoryOrderRepository.php          # (Adapter for Persistence port - Test)
    │   │
    │   └── Notifications/
    │       ├── EmailOrderNotification.php          # (Adapter for Notification port - Email)
    │       └── LogOrderNotification.php            # (Adapter for Notification port - Fake)
    │
    └── Providers/
        └── HexagonBindings.php                  # (Đăng ký DI cho các port)
```

Trong HexagonBindings.php (một Service Provider), ta thiết lập: `$this->app->bind(OrderRepository::class, EloquentOrderRepository::class);` `$this->app->bind(OrderNotification::class, EmailOrderNotification::class);`

Trong môi trường test, ta có thể override binding:

```
$this->app->bind(OrderRepository::class, InMemoryOrderRepository::class);
$this->app->bind(OrderNotification::class, LogOrderNotification::class);
```

Như vậy, core (PlaceOrderHandler) khi chạy sẽ được inject các adapter tương ứng tùy ngữ cảnh (thực tế hoặc test).

Xem qua PlaceOrderHandler (Application service) sau khi refactor:

```
class PlaceOrderHandler
{
    public function __construct(
        private OrderRepository $orderRepo,
        private ProductRepository $productRepo,
         private OrderNotification $notification // Giờ phụ thuộc interface
thay vì Mail trực tiếp
    ) {}

    public function handle(PlaceOrderCommand $cmd): OrderId
    {
        // ... [tạo Order, thêm sản phẩm, confirm như trước] ...

        $this->orderRepo->save($order);

          // Gửi thông báo qua cổng Notification (không biết chi tiết gửi
email)
        $this->notification->sendOrderConfirmation($order);

        return $order->id();
    }
}
```

OrderController (adapter cho UI port) không thay đổi nhiều, vẫn nhận request và gọi handler như cũ. EmailOrderNotification (adapter cho notification port) có thể dùng Mailable của Laravel:

```
class EmailOrderNotification implements OrderNotification
{
    public function sendOrderConfirmation(Order $order): void
    {
        // Sử dụng Mailable hoặc Mail facade để gửi email
```

```
        Mail::to($order->customerEmail())
            ->send(new OrderConfirmationMail($order));
    }
}
```

Còn LogOrderNotification (adapter giả) đơn giản ghi log:

```
class LogOrderNotification implements OrderNotification
{
    public function sendOrderConfirmation(Order $order): void
    {
                \Log::info("Fake send order confirmation for Order {$order->id()}");
    }
}
```

Kịch bản thử nghiệm: Nhờ cấu trúc port/adapter, ta dễ dàng viết test cho PlaceOrder: Thay vì dùng DB thật, ta dùng InMemoryOrderRepository: implement đơn giản lưu vào một mảng PHP . Thay vì gửi email thật, ta dùng LogOrderNotification để không gửi thật, chỉ ghi log. - Inject hai thứ giả này vào handler:

```
$handler = new PlaceOrderHandler(
    new InMemoryOrderRepository(),
      new InMemoryProductRepository($fakeProducts), // Giả lập danh sách product
    new LogOrderNotification()
);

$command = new PlaceOrderCommand(
    $customerId,
    [ /* ... danh sách sản phẩm ... */ ]
);
```

```
$orderId = $handler->handle($command);

// assert: kiểm tra đơn hàng đã lưu trong InMemoryOrderRepository
// assert: kiểm tra log có thông báo gửi email (nghĩa là OrderNotification
đã được gọi)
```

Test sẽ chạy rất nhanh, không cần boot Laravel, không kết nối mạng hay DB. Lại kiểm soát được hoàn toàn đầu ra (thông qua log hoặc dữ liệu trong repository giả). Đây chính là lợi ích thấy rõ của kiến trúc Ports & Adapters: test được phần lõi mà không phụ thuộc hạ tầng.

Bên cạnh đó, giả sử bây giờ team muốn thêm một cách mới để khách đặt hàng: qua một file CSV tải lên chẳng hạn (không qua UI web). Với kiến trúc này, ta chỉ việc viết một adapter mới (ví dụ một Console Command `ImportOrderCommand` đọc file CSV, rồi gọi `PlaceOrderHandler` cho mỗi dòng dữ liệu). Không cần đụng chạm gì đến domain hay application logic có sẵn. Tương tự, nếu muốn lưu order vào hai cơ sở dữ liệu khác nhau (phục vụ mục đích song song trong giai đoạn chuyển đổi chẳng hạn), ta có thể implement thêm một adapter repository nữa và cấu hình sao cho cả hai cùng nhận sự kiện (hoặc gọi tuần tự hai repository). Khả năng mở rộng rất linh hoạt.

Cuối cùng, lưu ý rằng kiến trúc hexagonal cũng rất hữu dụng khi tích hợp giữa các Bounded Context (bối cảnh miền) khác nhau trong Domain-Driven Design. Giả sử context "Đặt hàng" của ta cần lấy thông tin từ context "Khách hàng" (quản lý hồ sơ khách). Thay vì gọi thẳng database của context khách (phá vỡ bounded context), ta định nghĩa một port CustomerContext ở context Order, với các phương thức cần thiết (vd `findCustomerById`). Implement adapter cho port này có thể là `CustomerContextHttpAdapter` gọi REST API của service Khách hàng . Nếu sau này thay đổi cách tích hợp (ví dụ gộp database hay dùng cách khác), chỉ adapter này cần đổi. Context Order vẫn giữ code sạch và không bị phụ thuộc cứng vào context khác.

Tổng kết

Hexagon Architecture mang đến một cấp độ linh hoạt cao trong thiết kế phần mềm:

- **Decouple tuyệt đối core và bên ngoài:** Mọi sự phụ thuộc đều được đảo ngược, core chỉ biết đến interface (port), toàn bộ chi tiết bên ngoài đóng gói trong adapter. Nhờ đó, core không bao giờ bị "kẹt" với một công nghệ cụ thể nào .
- **Thử nghiệm dễ dàng, tăng chất lượng:** Bạn có thể chạy toàn bộ ứng dụng lõi trong môi trường giả lập một cách trơn tru - ví dụ chạy use case trong unit test sử dụng in-memory adapters thay cho I/O thật - đạt tốc độ nhanh và cô lập để tìm bug. Đồng thời, kiến trúc này vẫn đề cao test tích hợp cho adapter thật: mỗi adapter quan trọng cần có integration test (VD: test riêng OrderRepository với MySQL, test Notification gửi email thật) để đảm bảo adapter hoạt động đúng với thế giới thực .
- **Thay đổi công nghệ không ảnh hưởng core:** Muốn chuyển từ gửi email sang gửi SMS? Chỉ cần implement adapter SMS cho port Notification, core không cần sửa. Muốn đổi cơ sở dữ liệu? Viết adapter mới cho port Persistence, core cũng không đổi gì. Thậm chí muốn thay framework web (giả sử chuyển từ Laravel sang Lumen hoặc ExpressJS Node) - hoàn toàn khả thi vì phần giao diện web chỉ là adapter, domain và application có thể chuyển sang dùng qua adapter khác. Điều này bảo vệ khoản đầu tư lớn nhất: logic nghiệp vụ không bị lỗi thời khi công nghệ thay đổi.
- **Hỗ trợ đa kênh, đa nền tảng:** Ứng dụng dễ dàng mở ra các cổng giao tiếp mới (thêm API cho đối tác, thêm ứng dụng mobile gọi vào, v.v.) mà không phải viết lại logic. Tương tự, ứng dụng có thể đồng thời ghi dữ liệu ra nhiều nơi (database SQL, data warehouse NoSQL, file) để phục vụ các mục đích khác nhau chỉ bằng việc gắn thêm adapter. Kiến trúc hexagonal thực sự chuẩn bị cho tương lai mở rộng.
- **Tư duy rõ ràng về ranh giới:** Bằng cách định danh các port, kiến trúc buộc kiến trúc sư phải nghĩ rõ: "Hệ thống của mình giao tiếp với bên ngoài qua những chỗ nào?". Điều này dẫn đến thiết kế tổng thể nhất quán và tránh bỏ sót. Các developer cũng dễ hình dung hơn khi làm việc: thêm một tính năng liên quan tích hợp = thêm/đổi adapter, chứ không chèn code bừa bãi.
- **Tính module và test khả năng tích hợp:** Mỗi adapter có thể được phát triển, deploy độc lập (trong trường hợp kiến trúc plugin). Điều này cũng hữu ích nếu sau này ta tách dần monolith thành các dịch vụ nhỏ: nếu monolith ban đầu đã phân tách qua cổng rõ ràng, việc tách adapter thành service độc lập (dùng network call thay vì call nội bộ) sẽ ít ảnh hưởng đến core.

Tất nhiên, kiến trúc hexagonal cũng đòi hỏi một trình độ tổ chức cao. Số lượng interface và class có thể tăng lên đáng kể so với cách viết "dính chặt" truyền thống. Việc đặt tên, quản lý các adapter cũng cần quy ước để không rối. Cũng như layered, nếu áp dụng hexagonal một cách lý thuyết mà không hiểu mục đích, có thể tạo ra hệ thống phức tạp không cần thiết. Do vậy, hãy sử dụng nó một cách hợp lý: tập trung vào những điểm mà sự thay đổi/ mở rộng là có khả năng hoặc những phần khó test để áp dụng port & adapter, không nhất thiết mọi thứ đều phải trừu tượng hoá quá mức.

Đến đây, chúng ta đã đi qua các kiến trúc hiện đại trong xây dựng ứng dụng web: từ các mẫu thiết kế nền tảng, đến kiến trúc phân lớp, và mở rộng thành kiến trúc hình lục giác. Tất cả đều nhằm hướng tới mục tiêu chung: giữ cho phần mềm linh hoạt, dễ thích ứng trước thay đổi, và duy trì được tính ổn định cao theo thời gian. Việc vận dụng nhuần nhuyễn những nguyên tắc này sẽ giúp bạn thiết kế những hệ thống "trưởng thành", nơi thay đổi tính năng không làm ta e ngại, và công nghệ mới có thể tích hợp mà không phải viết lại từ đầu.

Lời kết

Bạn đã đọc hết cuốn “Kiến trúc ứng dụng Web” của mình.

Từ đáy lòng, mình cám ơn bạn rất nhiều vì đã đọc đến cuối quyển Ebook này. Có thể cách viết mình còn lộn xộn, mình mong bạn hiểu những tâm huyết của mình, và những mong mỏi được chia sẻ kiến thức đến cộng đồng. Thiết kế ứng dụng là điều rất thú vị, và mình mong muốn mọi người đều có những giải pháp tốt nhất cho ứng dụng của mình. Mình hy vọng ebook này sẽ giúp đỡ bạn phần nào trên con đường nâng cấp và cải thiện kỹ năng, trở thành một kiến trúc sư ứng dụng đúng nghĩa.

Cám ơn bạn rất nhiều !

Nếu bạn thích những gì trong cuốn sách này và muốn trao đổi thêm, đừng ngần ngại liên lạc với mình qua:

Email: huynt57@gmail.com

Facebook: Tại đây

Cuốn Ebook này sẽ không tồn tại nếu không có vợ mình Dương Thị Thu Huyền và con trai mình, cố vấn tí hon Nguyễn Dương Hoàng Khôi. Cám ơn gia đình đã luôn bên cạnh và ủng hộ mình.

Happy Coding !